Industry Classification

Code explanations, worked calculations, and oral-exam preparation

16 October 2026

Contents

How to use this guide **4**
What has actually been achieved . . . 4

Requirements and evidence **4**
What was deliberately kept simple . . . 5

Foundations: the objects being manipulated **5**
Variables, tables, arrays, and functions . . . 5
Features, targets, and embedding meaning . . . 6
The data objects after preparation . . . 6

Code block 01: Imports and resources **7**
Purpose and choices . . . 7
Numbered code references . . . 7

Code block 02: Loading Data **8**
What the financial table contributes . . . 8
Numbered code references . . . 8

Code block 03: Separating the companies **9**
Four roles, four disjoint groups . . . 9
Numbered code references . . . 9

Code block 04: Transforming Data (Label Encoding) and class distribution **11**
Category codes are names, not measurements . . . 11
Numbered code references . . . 11

Code block 05: PCA — dimensionality reduction and visualization **12**
PCA mathematics and interpretation . . . 12
Numbered code references . . . 12

Code block 06: Fitting and comparing Classifier Models **14**
Logistic regression . . . 14
k-nearest neighbors . . . 14
The ensemble weight . . . 14
Numbered code references . . . 15
The voting-weight result . . . 15

Code block 07: Classification results and errors **16**
Accuracy, precision, recall, and F1 . . . 16
Reading the confusion matrices . . . 16
Shared errors . . . 16
Numbered code references . . . 16

Code block 08: Optional: Confidence Weighted Prediction — official score **20**
What the official score measures . . . 20
Numbered code references . . . 20

Code block 09: Financial distributions **22**
Distribution analysis rather than automatic deletion . . . 22
Numbered code references . . . 22

Code block 10: Text votes and financial compatibility **26**
Financial compatibility, step by step . . . 26
Which industry's distribution is used? . . . 26
Why this is not a p-value . . . 26
Numbered code references . . . 27

Code block 11: Learning confidence **28**
A second prediction problem . . . 28
Numbered code references . . . 28

Code block 12: Evaluating confidence **29**
The comparison that can support a financial claim . . . 29
Numbered code references . . . 29
Actual confidence results . . . 29

Code block 13: Confidence and human review **30**
Calibration diagrams . . . 30
Review policy and its denominator . . . 30
Numbered code references . . . 30
Occupied confidence bins . . . 31

Code block 14: Predicting new companies and checking repeated representations **32**
The prediction contract . . . 32
Repeated representations and limits . . . 32
Numbered code references . . . 32

Worked calculations using the actual fitted model **34**
The learned correctness equation . . . 34
Case A: correct prediction . . . 34
Case B: incorrect prediction . . . 35
A class-level F1 calculation: Banks under LR . . . 35
The complete official score . . . 36

Oral-exam questions and model answers **36**
Project and data . . . 36
Splitting and leakage . . . 37
Labels, PCA, and model choices . . . 38
Evaluation and the official target . . . 39
Financial evidence and confidence . . . 40
Review, software, and reproduction . . . 42

A five-minute oral presentation **45**
Self-test without looking at the answers . . . 45

Reproduction, resources, and data provenance **46**
Running the analysis . . . 46
Memory arithmetic and benchmark scope . . . 46
Verification performed . . . 46
What belongs in GitHub . . . 46

Development checks and alternatives **47**

Complete per-industry results **47**

Financial reference observation counts **48**

Complete code listing **49**
Block 01: Imports and resources . . . 49
Block 02: Loading Data . . . 49

Block 03: Separating the companies 49
Block 04: Transforming Data (Label Encoding) and class distribution 49
Block 05: PCA — dimensionality reduction and visualization 50
Block 06: Fitting and comparing Classifier Models 50
Block 07: Classification results and errors 50
Block 08: Optional: Confidence Weighted Prediction — official score 51
Block 09: Financial distributions 51
Block 10: Text votes and financial compatibility 51
Block 11: Learning confidence 52
Block 12: Evaluating confidence 52
Block 13: Confidence and human review 52
Block 14: Predicting new companies and checking repeated representations 53

Sources and further reading **54**
Assignment and project sources 54
Official library documentation 54

How to use this guide

This guide accompanies `industry_classification.ipynb`. It explains the actual implementation, its measured results, the mathematics behind them, and the decisions that a student should be able to defend. Code comments such as **[10.18]** identify a particular operation. Search for the same number in this PDF or in the notebook. The number before the dot identifies the code block, and the number after it identifies the operation within that block.

Read the project overview first, then study the code blocks in order. Use the worked cases to connect individual operations into a complete prediction. Finally, practise the oral-exam questions without looking at the answers. The complete code listing at the end permits study away from Jupyter. A correct explanation should distinguish a measured fact, a modeling choice, a simplifying assumption, and an unverified claim.

The notebook contains the implementation and concise observations. Detailed teaching material belongs here. Both documents are in English. AI assistance was used to develop and review the code and explanatory material, as permitted by the student's stated course rules; this does not replace understanding the submitted work.

What has actually been achieved

- The supplied training table contains **31,706 records, 31,705 unique company IDs, 25 industries, and 768-dimensional embeddings**. The brief's reference to more than 40,000 companies does not change the number in the uploaded training subset.
- The required logistic-regression and k-NN classifiers, PCA scatterplot, class-frequency table, per-industry reports, both confusion heatmaps, and shared-error analysis are implemented.
- The optional approach combines classifier probabilities and estimates correctness confidence using text evidence and financial compatibility. It reports the official score, a calibration diagram, and an explicit human-review policy.
- Final-validation ensemble accuracy is **78.62%**, weighted F1 is **78.09%**, and the official score is **67.50%**.
- **The 85% score target was not reached locally.** Private-test performance is unknown. High-confidence-subset accuracy is not a substitute for the official whole-set score.
- The full cloud execution, including verification, took **221.1 seconds**. The measured PCA/transformation/classifier-fitting portion took **44.3 seconds**. The Intel Mac's 45-minute limit has not been directly tested.

These limitations are part of an accurate project defense. A shorter implementation does not justify inventing a stronger result.

Requirements and evidence

The current-year `code_skeleton_industry_classification.ipynb` supplies the assignment tasks and official scoring function. Previous-year work is background material; its different confidence-weighted calculation cannot replace this year's formula. The 85% aspiration, 45-minute laptop constraint, and request for a financial ensemble appear in the student's project brief. The skeleton does not independently establish a bonus-point award, deadline, or mandatory private-test function name.

Requirement	Where it is addressed
Use supplied business-description embeddings	[03.09]–[03.13]
Encode targets and check held-out classes	[04.01]–[04.03]
Show under/overrepresented classes	[04.04]–[04.05], counts table
Fit PCA on appropriate training data	[05.02]–[05.04]
Plot two components, color industries, label axes	[05.06]–[05.11]
Fit logistic regression and k-NN	[06.01]–[06.05]
Accuracy/F1 comparison and per-industry errors	[07.01]–[07.06]
Confusion heatmap for each model	[07.07]–[07.12]
Common confusions and shared errors	[07.13]–[07.17]
Financial distributions, missing values, outliers	[02], [09], [10]

Requirement	Where it is addressed
Ensemble and statistically justified confidence	[06.08]–[06.12], [10]–[12]
Confidence versus accuracy and uncertain cases	[13]
Tuple of label/confidence vectors	[14.01]–[14.04]
Exact current-year scoring formula	[08]

What was deliberately kept simple

There are no custom estimator classes, model-loading frameworks, deployment services, external embedding downloads, GPUs, or large neural networks. A short splitting function avoids repeating the same group-split operation. An embedding function is reused for fitting data and new companies. A single confidence-feature function prevents training and prediction from calculating different inputs. Other operations stay directly in the notebook.

Class frequencies are a table rather than an additional chart because the skeleton explicitly permits either. One loop produces the two classifiers' reports and confusion plots. Their correctness cross-table replaces several separate error-count statements. A dictionary iteration replaces an unnecessarily repeated nested lookup in financial scoring. Financial profiles use one record per company, and an unused small-sample branch has been removed after checking that every profile has at least 66 usable observations. No arbitrary rank smoothing is needed because compatibility can safely equal zero.

The implementation has **161 computational Python statements**, compared with **208** in the earlier full notebook when documentation strings are excluded. This is a comparison of syntax-tree statements, not a misleading count achieved by joining unrelated instructions with semicolons. Explanatory comments add visible lines without adding computations. Fixed library defaults are used unless there is a reason to change them.

Foundations: the objects being manipulated

Variables, tables, arrays, and functions

A **variable** is a name referring to a value. `df` refers to a pandas **DataFrame**, a table with named columns. `df.industry` selects its industry column, a **Series** with an index. A row is a supplied company record. A company can have more than one record, so rows and unique companies must not be confused.

A NumPy **array** stores numerical values with a shape. A **vector** has one dimension, such as a target array of shape `(3171,)`. A **matrix** has rows and columns, such as validation embeddings of shape `(3171, 768)`. In this project, axis 0 runs down companies and axis 1 runs across features or class probabilities. `argmax(1)` chooses a column separately for each company. `[0]` selects the first item because Python indexing begins at zero.

A **function** defines a reusable calculation. `def` begins the definition, indentation marks its body, and `return` supplies results to its caller. Returning two arrays creates a **tuple**. `labels, confidence = predict_industry(companies)` unpacks that tuple. Functions are used here where a real operation is repeated, rather than to create an unnecessary software architecture.

`=` assigns a value; `==` compares values. A comparison produces Boolean values, `True` and `False`. A Boolean **mask** selects matching rows. For arithmetic, `True` behaves like one and `False` like zero, so a correctness mask's mean is accuracy. `&` combines masks element by element; Python's `and` is not the corresponding array operation.

Lists preserve an order. A **list comprehension**, such as `[le.transform(part.industry) for part in parts]`, performs the same operation for each item and collects the results. Here `parts` always has the order fitting, tuning, calibration, validation. Tuple unpacking must use that same order. This is ordinary repetition removal; it does not change the statistical procedure.

Features, targets, and embedding meaning

A **feature** is information supplied to a model. The **target** is what the model learns to predict. The text features are 768 numbers representing a company's business description. The target is its known industry. An individual coordinate is not directly labeled "banking" or "manufacturing"; the coordinates jointly encode information produced by a pretrained text model.

The industry target is never inserted into the embedding matrix or read by the public prediction function. The company ID links tables and defines groups for splitting; it is not a numerical predictor. Country codes are not used in this solution. The financial ratios support confidence, rather than supplying the model with the true industry at prediction time.

An embedding is stored as a string containing a numerical list, rather than as a ready-to-fit numerical matrix. `ast.literal_eval` parses literal data. General `eval` could execute arbitrary code and is unnecessary. `float32` uses four bytes per coordinate. Fitting embeddings alone therefore require $25,681 \times 768 \times 4 = 78{,}892{,}032$ bytes; the MiB conversion is reported in the resource appendix. Parsing creates temporary Python objects, so the array's size is not the entire memory footprint.

The data objects after preparation

Object	Rows	Columns or entries	Meaning
`X_fit`	25,681	768	Original fitting embeddings
`X_tune`	1,427	768	Tuning embeddings
`X_cal`	1,427	768	Confidence-calibration embeddings
`X_val`	3,171	768	Final-validation embeddings
`P_fit`	25,681	128	Fitting PCA scores
`lr_val`, `knn_val`	3,171	25	Class probabilities
`features_cal`	1,427	5	Inputs to correctness confidence
`y_val`, `pred_val`, `confidence_val`	3,171	One-dimensional	True codes, predicted codes, confidence

`X` and `P` contain features, while `y` contains target codes. Predicted industry names are obtained through the same label encoder's inverse mapping. A row's position must remain consistent across all these objects; matching lengths alone would not prove correct alignment.

Code block 01: Imports and resources

Purpose and choices

Imports make established library implementations available. `as np`, `as pd`, `as plt`, and `as sns` are conventional shorter names, not separate algorithms. pandas handles tables and joins; NumPy handles arrays; scikit-learn supplies PCA, classifiers, target encoding, splitting, and metrics. Matplotlib supplies figures and axes; seaborn draws standard statistical plots on those axes. `display` produces readable Jupyter tables.

`threadpool_limits(2)` limits threads used by supported numerical libraries. k-NN also uses `n_jobs=2`, which controls its own parallel work. These settings do not prove that the entire process can never use more than two threads: different parallel layers can interact. They are modest resource choices for a laptop-oriented project, not a substitute for measuring execution.

`perf_counter()` reads a monotonic elapsed-time clock. Subtracting two readings measures a duration, not a calendar timestamp. The final notebook reading excludes the earlier import time and external verification. The external runner measurement includes kernel startup and the additional checks. Keeping those definitions separate prevents apparently contradictory timings.

No installation command runs inside the analysis. The reproducibility appendix gives tested versions. PDF-generation tools are not dependencies of the submitted classifier.

Numbered code references

- **[01.01]** `import ast` — Parse embedding strings safely as literal values.
- **[01.02]** `from time import perf_counter` — Measure elapsed time with a monotonic clock.
- **[01.03]** `import numpy as np` — Use numerical arrays and vector operations.
- **[01.04]** `import pandas as pd` — Load and join labeled tables.
- **[01.05]** `import matplotlib.pyplot as plt` — Create figures and label their axes.
- **[01.06]** `import seaborn as sns` — Draw the scatterplot, heatmaps, and boxplots.
- **[01.07]** `from IPython.display import display` — Display readable tables in Jupyter.
- **[01.08]** `from sklearn.decomposition import PCA` — Import the PCA implementation.
- **[01.09]** `from sklearn.linear_model import LogisticRegression` — Use the same established estimator for industry and correctness models.
- **[01.10]** `from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, f1_score` — Import the required classification and scoring tools.
- **[01.11]** `from sklearn.model_selection import GroupShuffleSplit` — Split company groups without separating repeated IDs.
- **[01.12]** `from sklearn.neighbors import KNeighborsClassifier` — Import the neighbor classifier.
- **[01.13]** `from sklearn.preprocessing import LabelEncoder` — Map industry names to integer class codes.
- **[01.14]** `from threadpoolctl import threadpool_limits` — Control numerical-library CPU threads.
- **[01.15]** `threadpool_limits(2)` — Limit numerical-library threads to two.
- **[01.16]** `started = perf_counter()` — Start timing after imports and thread setup.

Code block 02: Loading Data

What the financial table contributes

The three numerical columns are net profit margin, EBITDA margin, and asset turnover. Their economic interpretation is respectively net income relative to revenue, EBITDA relative to revenue, and revenue relative to assets. The supplied file contains ratios, not the original accounting denominators; the code does not claim to recompute those ratios from financial statements or to resolve accounting-period differences.

The raw financial file has 29,561 rows and 29,399 unique IDs. There are 162 repeated-ID rows, including 14 wholly duplicated rows. Raw missing counts are 1,111 for net profit margin, 19,279 for EBITDA margin, and 1,363 for asset turnover. Infinite counts are 2,131, 824, and 5. NaN and infinity are counted separately before infinity is replaced. A finite negative margin can indicate a genuine loss and is retained.

Repeated records are collapsed using a separate median for each ratio within each company. This is a robust summary and avoids choosing an arbitrary first row. It is not a median across different companies and does not use industry labels. The file does not supply enough time information to justify claiming that the selected median is a specific reporting year's value. This aggregation loses distinctions between a company's records, an explicit tradeoff.

`groupby('id')[ratios].median()` produces one row per ID with the ID as index. `merge(..., on='id', how='left')` links that index to the main ID column. A left join retains every main record, including companies without a financial match. `validate='many_to_one'` checks that multiple main records cannot match several financial records and unexpectedly multiply rows. There are financial matches for 29,400 main records, but a matched record can still have missing individual ratios.

This cleaning can precede the company split because each aggregation uses only records of that same ID and no targets; it does not estimate a global imputation value from validation data. Industry reference distributions are different: they use targets and are therefore constructed only from fitting companies.

Numbered code references

- **[02.01]** `ratios = ['net_profit_margin', 'ebitda_margin', 'asset_turnover']` — Select the three supplied numerical financial ratios.
- **[02.02]** `financials = pd.read_pickle('data/df_financials_train.pkl')` — Read the supplied financial table.
- **[02.03]** `display(pd.DataFrame({'Missing': financials[ratios].isna().sum(), 'Infinite': np.isinf(financials[ratios]).su...` — Count missing and infinite values before cleaning.
- **[02.04]** `print('Repeated financial IDs:', financials.id.duplicated().sum())` — Check how many financial rows repeat a company ID.
- **[02.05]** `financials = financials.replace([np.inf, -np.inf], np.nan).groupby('id')[ratios].median()` — Treat infinity as missing; take one median per ratio and company.
- **[02.06]** `df = pd.read_pickle('data/df_train.pkl').merge(financials, on='id', how='left', validate='many_to_one')` — Keep every main record and attach at most one financial row per ID.

Code block 03: Separating the companies

Four roles, four disjoint groups

Fitting estimates classifier parameters, PCA directions, and financial reference distributions. Tuning selects the voting weight. Calibration estimates correctness confidence after the base classifiers and vote are fixed. Validation evaluates the resulting system. Reusing calibration labels to choose voting weights would let a choice depend on the same correctness targets used to teach confidence; a separate tuning subset avoids that dependence.

Starting with all IDs, the first split reserves 10% for validation. The second holds out 10% of the remaining 90%, giving approximately 81% fitting and 9% other development. The third divides that 9% equally, producing 4.5% tuning and 4.5% calibration:

$$0.90 \times 0.90 = 0.81, \qquad 0.90 \times 0.10 \times 0.50 = 0.045.$$

Actual row counts are 25,681, 1,427, 1,427, and 3,171. Percentages are approximate because the splitter operates on unique IDs and one company has two different embedding records. That company's two records have the same target and are kept together; there are 25,680 unique fitting IDs. Removing them both would discard valid information, while a random row split could put records of the same company on both sides.

`GroupShuffleSplit` returns a generator of position pairs. `n_splits=1` requests one pair, and `next(...)` retrieves it. `.iloc` selects positions within the particular frame being split. This avoids confusing frame index labels with row positions. The fixed seed makes the ID splits repeatable. It does not stratify industry proportions, so the code checks class coverage rather than pretending stratification was used.

Embedding conversion itself is a deterministic per-record parsing operation, not a learned transformation. It would not leak targets merely by parsing all records, but keeping the four arrays explicit makes their roles and row order easy to inspect. PCA and the classifiers are fitted only on `X_fit`; the remaining arrays are transformed or predicted without refitting.

The labels of the private test set have never been available. The local validation set was examined in earlier project versions, so this revision's numbers are reused-holdout results, not a newly collected independent test. The simplification does not justify a fresh generalization guarantee.

Numbered code references

- **[03.01]** `def split(companies, size):` — Reuse one short company-group splitting function.
- **[03.02]** `splitter = GroupShuffleSplit(n_splits=1, test_size=size, random_state=42)` — Choose one reproducible group split of the requested size.
- **[03.03]** `left, right = next(splitter.split(companies, groups=companies.id))` — Take its two arrays of row positions.
- **[03.04]** `return companies.iloc[left], companies.iloc[right]` — Select those rows without changing their order.
- **[03.05]** `df_development, df_val = split(df, 0.1)` — Reserve about ten percent of company IDs for validation.
- **[03.06]** `df_fit, df_other = split(df_development, 0.1)` — Separate model fitting from the remaining development records.
- **[03.07]** `df_tune, df_cal = split(df_other, 0.5)` — Divide the held-out development records into tuning and calibration.
- **[03.08]** `parts = [df_fit, df_tune, df_cal, df_val]` — Keep one consistent order for all feature and label sets.
- **[03.09]** `def embeddings(companies):` — Reuse the embedding conversion for existing and new companies.
- **[03.10]** `return np.array(companies.business_description_embedding.apply(ast.literal_eval).tolist(), dtype=np.float32)` — Parse the strings and store their coordinates as float32 arrays.

- **[03.11]** `X_fit, X_tune, X_cal, X_val = [embeddings(part) for part in parts]` — Convert each of the four sets once.
- **[03.12]** `print('Fitting, tuning, calibration, validation records:', [len(part) for part in parts])` — Show the actual record counts rather than assumed percentages.
- **[03.13]** `print('Embedding coordinates:', X_fit.shape[1])` — Show how many coordinates each embedding contains.

Code block 04: Transforming Data (Label Encoding) and class distribution

Category codes are names, not measurements

`LabelEncoder` sorts the fitting industry names and assigns integer codes from 0 to 24. The code is an identifier: industry 4 is not twice industry 2. scikit-learn classifiers expect a one-dimensional vector of categorical targets; one-hot encoding the targets is unnecessary here. The mapping is fitted once and reused for all four target arrays and for decoding predicted names.

The subset assertion checks that every tuning, calibration, or validation industry appears in fitting data. It meets the skeleton's class-coverage task and prevents silently using incompatible encodings. Applying a separate encoder to each set would risk assigning different meanings to the same integer or omitting a category in a smaller set.

Class frequencies are calculated from the actual fitting records. Materials has 6,797 records, or 26.47%, while Consumer Staples Distribution & Retail has 151. Their ratio is about 45.0. The frequency table divides each count by 25,681; its proportions sum to one. It does not use validation class supports as fitting weights.

The table is sufficient because the assignment permits either a frequency table or a plot. Large classes have more examples to learn from and more influence on accuracy and support-weighted F1. Small-class results can be noisy. Nevertheless, sample size is not a complete explanation of difficulty: Commercial & Professional Services has hundreds of fitting examples and still performs poorly. Class weighting or oversampling should not be added automatically; they change the training objective and can hurt the specified weighted-F1 objective.

Numbered code references

- **[04.01]** `le = LabelEncoder().fit(df_fit.industry)` — Learn the industry-code mapping from fitting records only.
- **[04.02]** `assert set(df_tune.industry).union(df_cal.industry, df_val.industry).issubset(le.classes_)` — Check the skeleton requirement that held-out classes are known.
- **[04.03]** `y_fit, y_tune, y_cal, y_val = [le.transform(part.industry) for part in parts]` — Apply the same mapping to every target vector.
- **[04.04]** `counts = df_fit.industry.value_counts()` — Count industries in the actual classifier-fitting set.
- **[04.05]** `display(pd.DataFrame({'Fitting records': counts, 'Share': counts / counts.sum()}))` — Show counts and proportions without an additional plot.

Code block 05: PCA — dimensionality reduction and visualization

PCA mathematics and interpretation

Let X be the fitting matrix and μ its mean vector. PCA centres the columns and finds orthonormal directions with maximum variance. If V_{128} contains the retained directions, a record is projected as

$$z = (x - \mu)V_{128}^{\mathsf{T}}.$$

`pca.mean_` has 768 entries and `pca.components_` has shape $(128, 768)$. The output has 128 coordinates per record. Both μ and the directions are learned from fitting records only. `fit_transform` learns and projects fitting data; later `transform` calls reuse the same fitted values. Fitting PCA on the whole dataset would allow evaluation features to influence the learned representation.

The variance explained by component j is its eigenvalue divided by the sum of all eigenvalues. Summing the first two gives 25.29%; summing all 128 gives 87.62%. These are proportions of feature variance, not classification accuracies or fractions of industry information recovered. A low-variance direction can still distinguish labels, so retaining variance is not an independent performance guarantee.

The scatterplot displays only the first two coordinates. Every dot is a fitting record, color represents the known industry, and transparency reveals overlapping regions. The upper pharmaceutical concentration suggests some industry-related structure, but many industries overlap centrally. The use of labels to color an already fitted unsupervised projection does not train PCA with labels. Overlap in two coordinates does not prove inseparability in 768 dimensions.

Logistic regression retains the full embeddings; k-NN uses PCA. Thus the comparison evaluates two practical pipelines on identical validation companies, rather than isolating an intrinsic causal advantage of one learning algorithm. A development comparison favored 128 components over 64 and over the tested raw-coordinate k-NN variant. Whitening, which rescales PCA coordinates to unit variance, weakened the tested ensemble. Extra scaling is therefore omitted.

The current solver choice is scikit-learn's default `auto`. With these dimensions and the tested version it uses a covariance-based route. The seed is supplied for reproducibility if a randomized route applies; a seed alone does not make every floating-point operation identical across operating systems.

Numbered code references

- **[05.01]** `fit_started = perf_counter()` — Start timing PCA preparation.
- **[05.02]** `pca = PCA(n_components=128, random_state=42)` — Keep 128 components using the fixed random seed.
- **[05.03]** `P_fit = pca.fit_transform(X_fit)` — Learn PCA from fitting embeddings and project them.
- **[05.04]** `P_tune, P_cal, P_val = [pca.transform(X) for X in [X_tune, X_cal, X_val]]` — Project the other sets without learning another PCA.
- **[05.05]** `fit_seconds = perf_counter() - fit_started` — Store the PCA and transformation time.
- **[05.06]** `fig, ax = plt.subplots(figsize=(10, 6))` — Create a figure and axes for the required scatterplot.
- **[05.07]** `sns.scatterplot(x=P_fit[:, 0], y=P_fit[:, 1], hue=df_fit.industry.to_numpy(),` — Color fitting points by industry; small transparent points show overlap.
- **[05.08]** `ax.set(xlabel='Principal component 1', ylabel='Principal component 2',` — Label the two PCA coordinates and the plot.
- **[05.09]** `ax.legend(bbox_to_anchor=(1.02, 1), loc='upper left', fontsize=7, markerscale=3)` — Place the long industry legend outside the data region.

- **[05.10]** `fig.tight_layout()` — Fit the labels and legend within the figure.
- **[05.11]** `plt.show()` — Display the scatterplot.
- **[05.12]** `print('Variance retained by 2 and 128 components:',` — Report the fractions of variance retained, not classification accuracy.

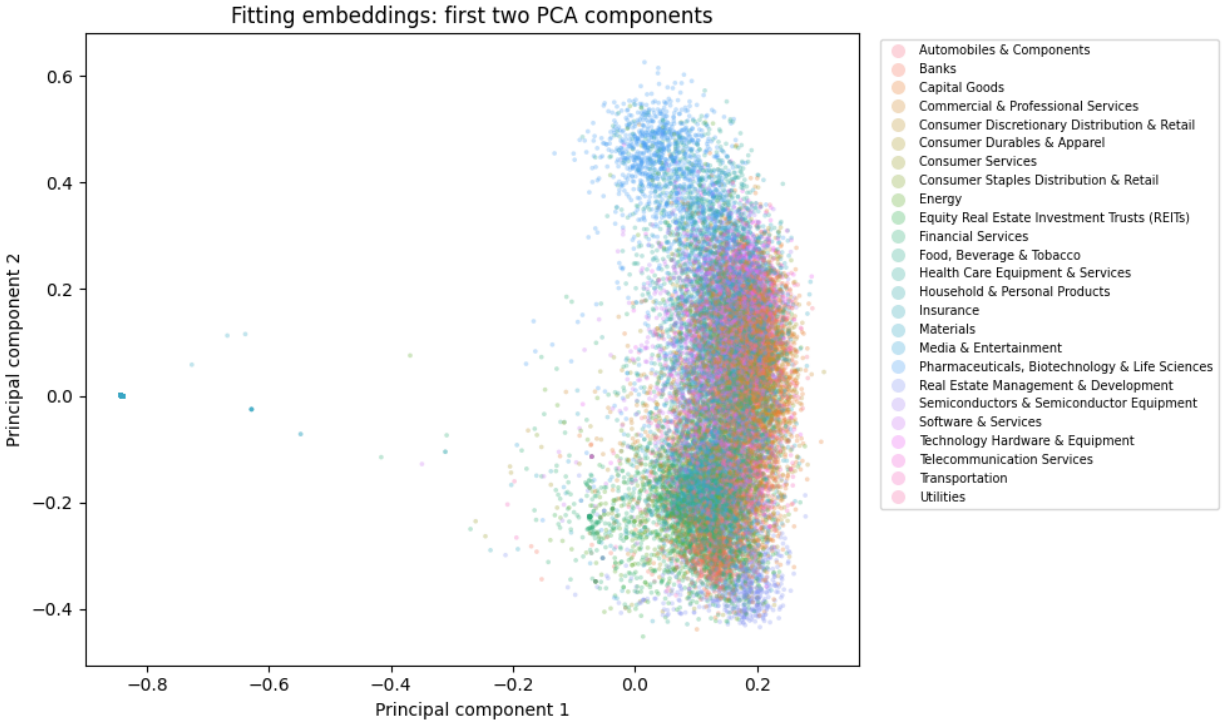


Figure 1: Figure 1. Fitting embeddings projected onto the first two PCA components. The upper pharmaceutical concentration and broad central overlap are visible; only 25.3% of variance is displayed.

Code block 06: Fitting and comparing Classifier Models

Logistic regression

Multiclass logistic regression is a classification model despite its name. It learns one linear score per industry and converts the scores into probabilities with softmax:

$$s_k(x) = w_k^{\mathsf{T}} x + b_k, \qquad p_k(x) = \frac{e^{s_k(x)}}{\sum_j e^{s_j(x)}}.$$

With no class/sample weights, the regularized objective is equivalent, up to multiplication by a positive constant, to

$$-\sum_{i=1}^{n} \log p_{y_i}(x_i) + \frac{1}{2C} \sum_k \|w_k\|_2^2.$$

The default L2 penalty discourages unnecessarily large weights; the intercept is not included in this penalty for the chosen solver. Larger C weakens regularization. `C=10` was useful in the ensemble, even though `C=1` was slightly stronger for LR alone in the development comparisons. `max_iter=1000` is a cap, not the number of passes the model must perform. The measured fit used 188 iterations. The default solver handles multinomial targets without a manual one-versus-rest loop.

k-nearest neighbors

k-NN retains fitting examples. For a query it finds the 15 nearest PCA records using Euclidean distance and assigns distance-weighted votes:

$$d(x,z) = \sqrt{\sum_j (x_j - z_j)^2}, \qquad p_k(x) = \frac{\sum_{i \in N_{15}(x)} w_i \, \mathbb{1}[y_i = k]}{\sum_{i \in N_{15}(x)} w_i}, \quad w_i = 1/d_i.$$

For exact zero distances, scikit-learn gives weight only to zero-distance neighbors, rather than dividing by zero in this naive formula. Ties and repeated representations can therefore produce very confident neighbor votes. These are model vote proportions, not automatically calibrated correctness probabilities.

Very small k can be sensitive to individual observations; very large k can mix different industries. The bounded checks included 5, 15, and 25 neighbors. Fifteen was the retained practical choice. Moving from 768 to 128 distance coordinates reduces the dimension of each distance calculation by a factor of six, but does not prove a sixfold wall-clock speedup.

The ensemble weight

Soft voting combines class-aligned probability columns:

$$v_{ik} = \alpha p_{ik}^{LR} + (1-\alpha) p_{ik}^{kNN}, \qquad \hat{y}_i = \arg\max_k v_{ik}.$$

The notebook compares $\alpha \in \{0.25, 0.50, 0.75\}$ on tuning weighted F1. It selects 0.25, so k-NN contributes 75% of each probability vector. This is not a class weight, a confidence exponent, or a sample weight. The probability rows still sum to one. The columns align because both classifiers are fitted with the same encoded target classes; verification checks their `classes_` arrays.

This small tuning table is reproducible within the notebook. The broader development checks are summarized later, rather than adding an exhaustive search to the submitted implementation. No calibration or validation label selects this weight in the current procedure.

Numbered code references

- **[06.01]** `lr = LogisticRegression(C=10, max_iter=1000)` — Use the development-selected LR regularization and iteration limit.
- **[06.02]** `knn = KNeighborsClassifier(n_neighbors=15, weights='distance', n_jobs=2)` — Use fifteen distance-weighted neighbors with two parallel jobs.
- **[06.03]** `fit_started = perf_counter()` — Start timing the two classifier fits.
- **[06.04]** `lr.fit(X_fit, y_fit)` — Learn industry weights from the original fitting embeddings.
- **[06.05]** `knn.fit(P_fit, y_fit)` — Store fitting neighbors in the PCA representation.
- **[06.06]** `fit_seconds += perf_counter() - fit_started` — Add classifier fitting time to the PCA preparation time.
- **[06.07]** `print('PCA, transforms, and classifier fitting seconds:', round(fit_seconds, 1))` — Report this measured part of the computation.
- **[06.08]** `lr_tune, knn_tune = lr.predict_proba(X_tune), knn.predict_proba(P_tune)` — Get both classifiers' probabilities on tuning companies.
- **[06.09]** `weights = [0.25, 0.5, 0.75]` — Compare three simple logistic-regression voting weights.
- **[06.10]** `tuning_f1 = [f1_score(y_tune, (w * lr_tune + (1 - w) * knn_tune).argmax(1), average='weighted') for w in weig...` — Score each probability mixture using tuning weighted F1 only.
- **[06.11]** `weight = weights[np.argmax(tuning_f1)]` — Keep the weight with the highest tuning score.
- **[06.12]** `display(pd.DataFrame({'LR weight': weights, 'Tuning weighted F1': tuning_f1}))` — Show the small tuning comparison.
- **[06.13]** `lr_cal, knn_cal = lr.predict_proba(X_cal), knn.predict_proba(P_cal)` — Cache calibration probabilities for the correctness models.
- **[06.14]** `lr_val, knn_val = lr.predict_proba(X_val), knn.predict_proba(P_val)` — Cache validation probabilities for all evaluations.
- **[06.15]** `predictions = {'Logistic regression': lr_val.argmax(1), 'k-NN': knn_val.argmax(1)}` — Take the highest-probability industry code for each base model.

The voting-weight result

LR weight	k-NN weight	Tuning weighted F1
0.25	0.75	0.791847
0.50	0.50	0.783686
0.75	0.25	0.782247

Code block 07: Classification results and errors

Accuracy, precision, recall, and F1

For one industry treated as the positive class, TP counts correct predictions of that industry, FP counts other industries incorrectly predicted as it, and FN counts its records assigned elsewhere:

$$\text{precision} = \frac{TP}{TP+FP}, \quad \text{recall} = \frac{TP}{TP+FN}, \quad F_1 = \frac{2PR}{P+R} = \frac{2TP}{2TP+FP+FN}.$$

Accuracy is the total fraction correct. Class support n_k is the number of validation records truly belonging to class k. The two reported F1 averages are

$$F_1^{weighted} = \sum_k \frac{n_k}{n} F_{1,k}, \qquad F_1^{macro} = \frac{1}{K} \sum_k F_{1,k}.$$

Weighted F1 is not the harmonic mean of globally averaged precision and recall. It weights already calculated per-class F1 values. In single-label multiclass classification, micro F1 equals accuracy; weighted and macro F1 generally do not. `zero_division=0` gives undefined per-class divisions a defined zero result in the report.

The actual LR weighted F1 is 77.78%, versus 77.14% for k-NN. LR accuracy is 78.15% and k-NN accuracy 77.70%. Their macro F1 scores are 71.75% and 71.05%. LR is the stronger required classifier on this holdout; that does not prove it will win on every population or split.

Reading the confusion matrices

Entry C_{ij} counts true class i predicted as class j. Row normalization divides by $\sum_j C_{ij}$. A diagonal normalized cell is therefore recall, not precision. A column-normalized diagonal would instead describe precision. Each row of these plots sums to one, giving rare industries visible weight in the display even though their estimates can be noisier.

The printed top pairs use raw counts after diagonal cells are set to zero. The figure has already been created from a separate normalized table, so this removal affects the error ranking, not the displayed heatmap. Capital Goods is predicted as Materials 28 times by LR and 33 times by k-NN. Technology Hardware & Equipment is predicted as Capital Goods 21 and 24 times. Related industry activities are a plausible explanation, but original description text is unavailable here, so counts alone cannot establish the precise reason.

The fitting-size/error-rate correlation is -0.362 for LR and -0.283 for k-NN. pandas aligns the industry-name indexes before calculating Pearson correlation. A negative result describes an association, not proof that sample size causes error. Label ambiguity, feature overlap, class definitions, and missing semantic detail can also matter.

Shared errors

Both models fail on 520 validation records. LR alone fails on 173, and k-NN alone on 187. Both are correct on 2,291. These four counts total 3,171. Their errors are highly overlapping, limiting the ensemble gain. The ensemble nevertheless reaches 78.09% weighted F1, about 0.31 percentage points above the stronger required classifier. Diversity helps only where the alternative model can correct a useful error without creating another one.

Numbered code references

- **[07.01]** `rows = []` — Collect the two models' summary metrics.
- **[07.02]** `for name, prediction in predictions.items():` — Analyze both classifiers with the same short loop.
- **[07.03]** `rows.append([name, accuracy_score(y_val, prediction),` — Calculate accuracy and weighted/macro F1 on the same records.

- **[07.04]** `report = pd.DataFrame(classification_report(y_val, prediction,` — Get per-industry precision, recall, F1, and support.
- **[07.05]** `print(name, '| fitting size / error-rate correlation:', round(counts.corr(1 - report['recall']), 3))` — Align industry names when relating fitting size to validation errors.
- **[07.06]** `display(report.sort_values('f1-score'))` — Show the weakest industries first.
- **[07.07]** `cm = pd.DataFrame(confusion_matrix(y_val, prediction, labels=np.arange(len(le.classes_))),` — Count true-industry rows against predicted-industry columns.
- **[07.08]** `fig, ax = plt.subplots(figsize=(11, 9))` — Create axes for this model's confusion heatmap.
- **[07.09]** `sns.heatmap(cm.div(cm.sum(axis=1), axis=0), cmap='Blues', vmin=0, vmax=1, ax=ax)` — Normalize each true-industry row before coloring it.
- **[07.10]** `ax.set(xlabel='Predicted industry', ylabel='True industry', title=name + ': fraction of each true industry')` — Label the true and predicted axes explicitly.
- **[07.11]** `fig.tight_layout()` — Keep the long industry labels visible.
- **[07.12]** `plt.show()` — Display the confusion heatmap.
- **[07.13]** `np.fill_diagonal(cm.values, 0)` — Exclude correct counts from the error-pair ranking.
- **[07.14]** `display(cm.stack().nlargest(5).rename('Misclassifications'))` — Show the five largest off-diagonal confusion counts.
- **[07.15]** `comparison = pd.DataFrame(rows, columns=['Model', 'Accuracy', 'Weighted F1', 'Macro F1']).set_index('Model')` — Build the direct classifier comparison table.
- **[07.16]** `display(comparison)` — Display the comparison.
- **[07.17]** `display(pd.crosstab(predictions['Logistic regression'] == y_val, predictions['k-NN'] == y_val,` — Count records where the models are correct or wrong together.

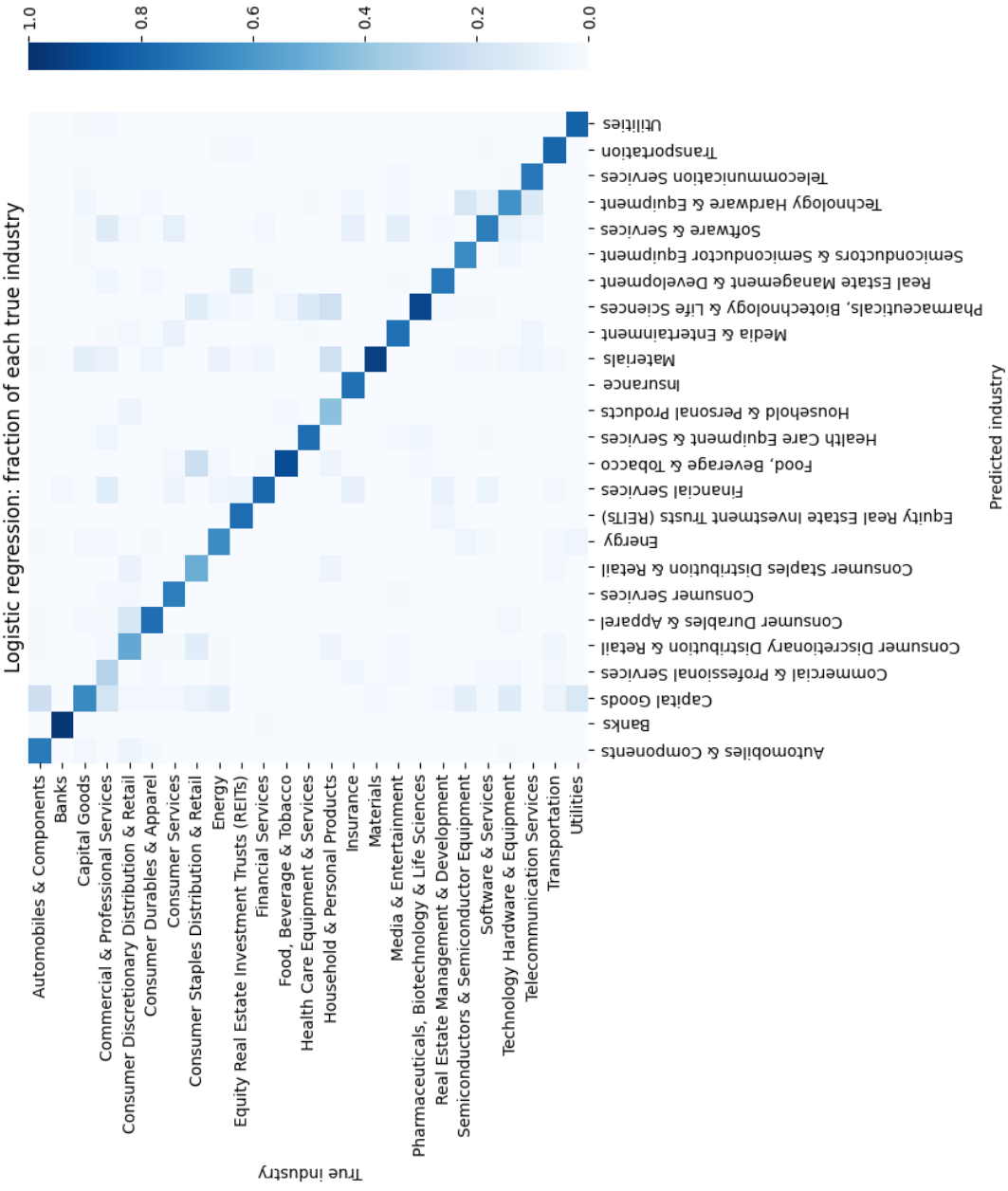


Figure 2. Logistic-regression row-normalized confusion matrix: the diagonal shows per-industry recall.

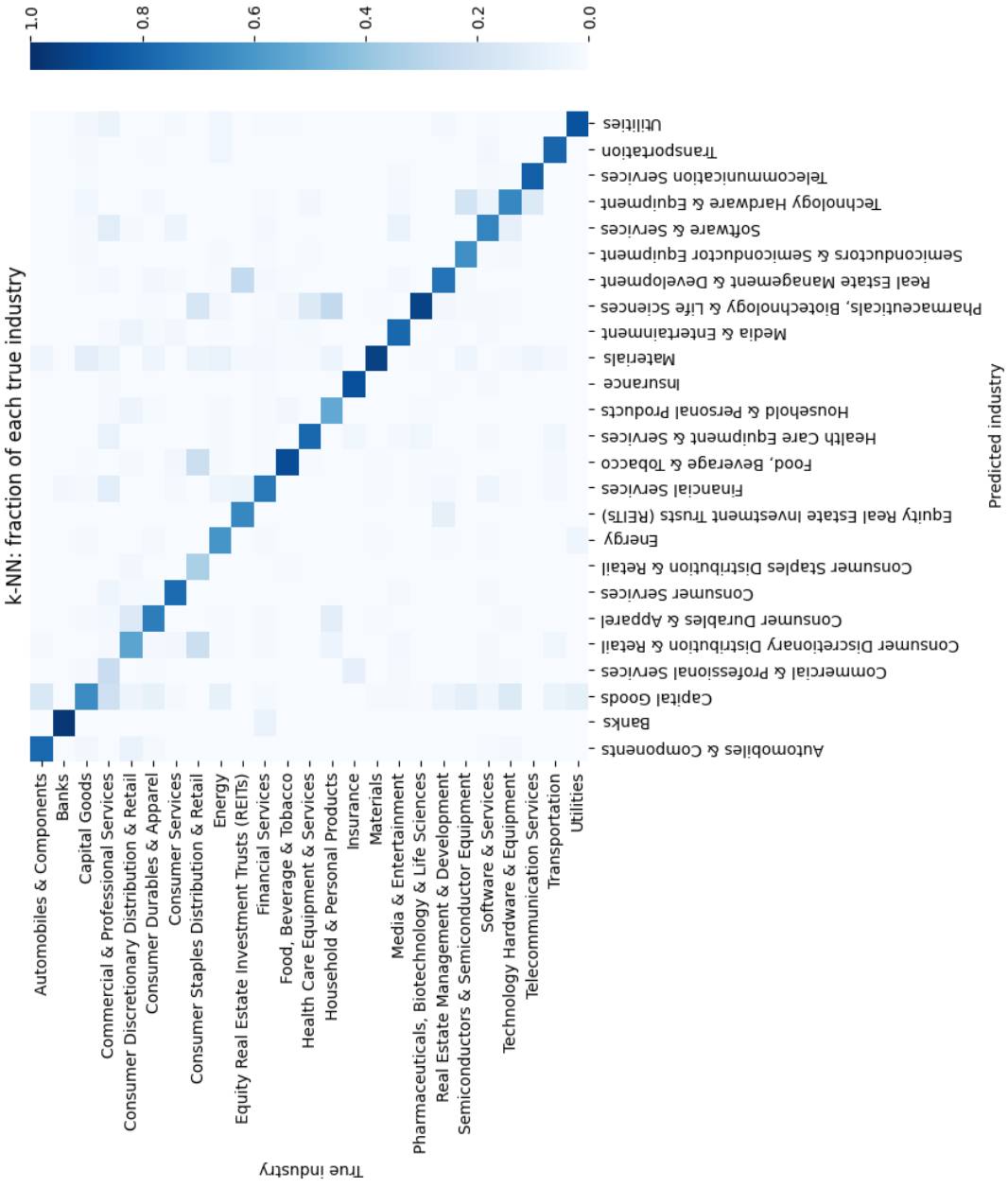


Figure 3. k-NN row-normalized confusion matrix, evaluated on the same 3,171 records.

Code block 08: Optional: Confidence Weighted Prediction — official score

What the official score measures

For fixed industry predictions define $z_i = \mathbb{1}[\hat{y}_i = y_i]$ and confidence $c_i \in [0, 1]$:

$$B = \frac{1}{n}\sum_{i=1}^{n}(c_i - z_i)^2, \qquad S = F_1^{weighted}(1 - B).$$

This is the current-year formula. Its Brier term concerns the probability that the chosen prediction is correct. It is not the full-vector multiclass Brier calculation, and confidence does not enter the class-support-weighted F1 calculation. The supplied function's input conversions, length/bounds checks, and numerical formula are retained; the lengthy documentation string is shortened because the detailed explanation is here.

A correct prediction at confidence 0.9 contributes $(0.9 - 1)^2 = 0.01$. An incorrect one at 0.9 contributes $0.9^2 = 0.81$. Reducing confidence everywhere is not a solution: a correct prediction at 0.1 contributes 0.81. Likewise, always reporting one makes each mistake cost one.

If the conditional probability of correctness is q, expected squared error is

$$q(c - 1)^2 + (1 - q)c^2 = (c - q)^2 + q(1 - q).$$

It is minimized at $c = q$. This explains why correctness probability is an appropriate target, rather than arbitrary shrinking or exponentiation. For fixed predicted labels, maximizing the official score is equivalent to minimizing this Brier term because F1 remains constant.

For the financial-calibrated ensemble the actual calculation is

$$S = 0.780881472\,(1 - 0.135581702) = 0.675008233.$$

Multiply by 100 to express this as 67.50%. Since $0 \le B \le 1, S \le F_1^{weighted}$. The current 78.09% F1 therefore makes an 85% score impossible for these validation predictions, even with perfect confidence. If F1 were 0.90, a score above 0.85 would require $B < 1 - 0.85/0.90 \approx 0.05556$. This is a joint classification-and-confidence objective, not an accuracy threshold.

The older solution's confidence-to-the-sixth transformation belongs to a different objective and offers no general guarantee here. A confidence of $0.9^6 \approx 0.5314$ reduces an incorrect prediction's penalty but increases a correct prediction's penalty. Whether the net effect helps depends on the events and cannot be asserted from the exponent alone.

Numbered code references

- **[08.01]** `def confidence_score(y_true, y_pred, confidence):` — Keep the course's confidence-score interface and formula.
- **[08.02]** `y_true = np.asarray(y_true)` — Allow true labels to be supplied as a list or array.
- **[08.03]** `y_pred = np.asarray(y_pred)` — Allow predicted labels to be supplied as a list or array.
- **[08.04]** `confidence = np.asarray(confidence, dtype=float)` — Use numerical floating-point confidence values.
- **[08.05]** `assert len(y_true) == len(y_pred) == len(confidence), 'Inputs must have the same length'` — Require one prediction and confidence for every target.
- **[08.06]** `assert np.all((confidence >= 0) & (confidence <= 1)), 'Confidence must be between 0 and 1'` — Require every confidence to lie between zero and one.
- **[08.07]** `f1 = f1_score(y_true, y_pred, average='weighted')` — Calculate class-support-weighted F1; confidence is not a weight.

- **[08.08]** `correct = (y_true == y_pred).astype(float)` — Represent a correct prediction as one and an error as zero.
- **[08.09]** `brier = np.mean((confidence - correct) ** 2)` — Average the squared errors of the correctness confidence.
- **[08.10]** `return f1 * (1 - brier)` — Apply exactly the official multiplicative score.

Code block 09: Financial distributions

Distribution analysis rather than automatic deletion

Financial references use 25,680 unique fitting company IDs, rather than counting the one repeated company twice. This deduplication is limited to economic references. Both distinct description records remain available to the text classifiers, and their common ID remains in a single split.

The summary shows counts and medians by industry. A boxplot shows the median and interquartile range; its conventional whiskers extend to observations within 1.5 times the IQR of the quartiles. Hiding the separately drawn outlier dots avoids compressing the display. It does not trim data before the reference distributions or confidence learning.

There is considerable overlap, particularly in net profit margins. Pharmaceuticals include many negative observations. Banks and REITs show low typical asset turnover compared with Consumer Staples Distribution & Retail. These observations support using finances as partial evidence, rather than as deterministic industry labels. Missingness is particularly strong for EBITDA margin and differs by industry.

`symlog` is a plotting transformation: it is linear near zero, within the chosen threshold of 0.1, and logarithmic in magnitude outside that region while preserving sign. An ordinary log scale cannot display zero or negative values. This axis transformation does not alter the numbers used by the classifier or financial percentile calculation. It must not be mistaken for evidence that the original data are normally or log-normally distributed.

No normality test is required to justify the empirical rank method. A large-sample normality test can reject harmless deviations, and a transformed distribution is not automatically Gaussian. The chosen calculation needs ordered finite reference observations, rather than a fitted Gaussian mean and standard deviation. Valid losses and large finite magnitudes are retained; infinities have no finite position and were marked missing earlier.

Numbered code references

- **[09.01]** `profile_data = df_fit.drop_duplicates('id')` — Give each fitting company one vote in the financial references.
- **[09.02]** `display(profile_data.groupby('industry')[ratios].agg(['count', 'median']))` — Show observation counts and medians for every industry and ratio.
- **[09.03]** `for column in ratios:` — Inspect all three ratios with the same plotting code.
- **[09.04]** `fig, ax = plt.subplots(figsize=(9, 7))` — Create readable axes for this ratio.
- **[09.05]** `sns.boxplot(data=profile_data, x=column, y='industry', showfliers=False, ax=ax)` — Show quartiles and medians; hide dots rather than delete outliers.
- **[09.06]** `ax.set_xscale('symlog', linthresh=0.1)` — Keep negative values while displaying large magnitudes.
- **[09.07]** `ax.set(xlabel=column.replace('_', ' '), ylabel='Industry',` — Label the financial ratio, industry, and plot title.
- **[09.08]** `fig.tight_layout()` — Prevent labels from being clipped.
- **[09.09]** `plt.show()` — Display this financial distribution.

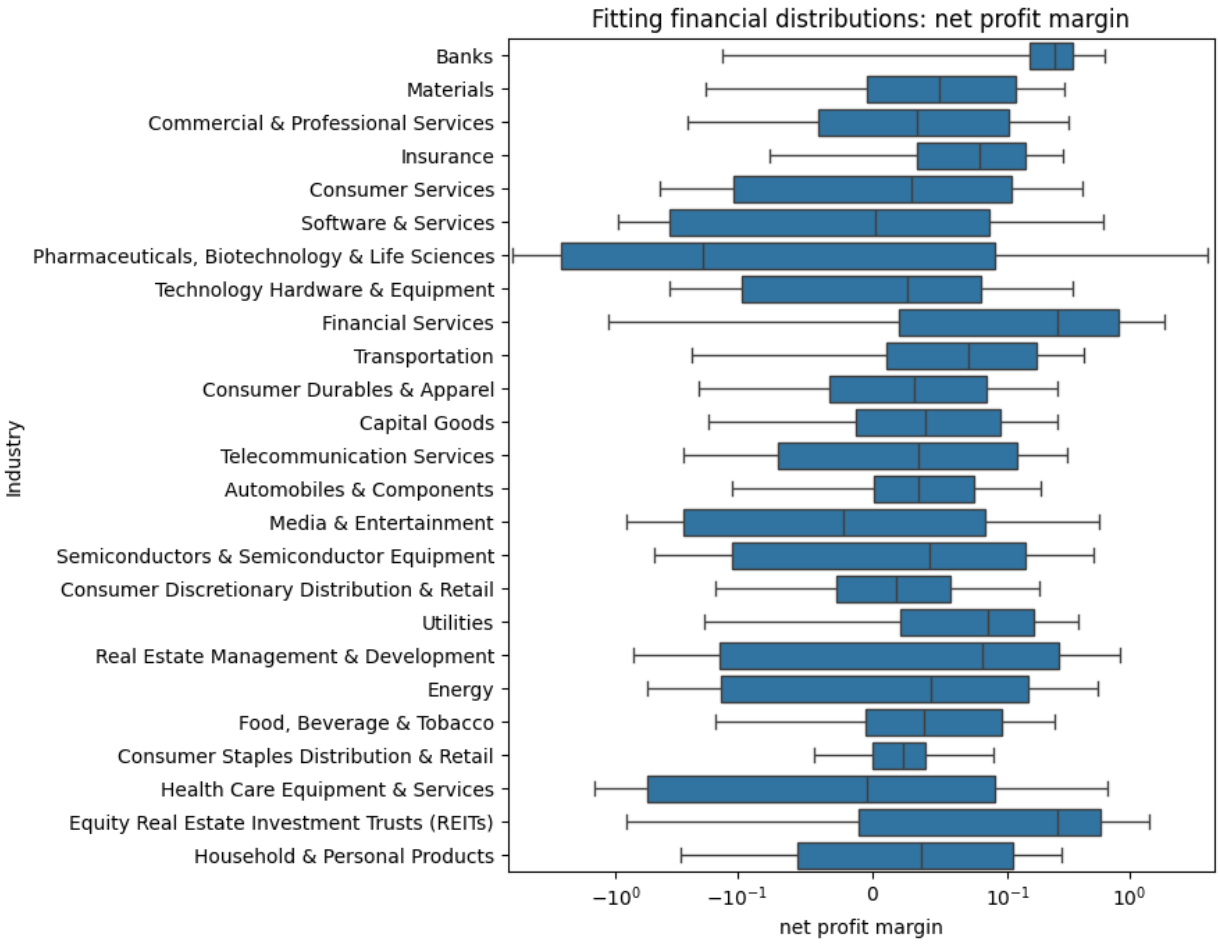


Figure 2: Figure 4. Net profit margin across unique fitting companies. Negative observations remain present.

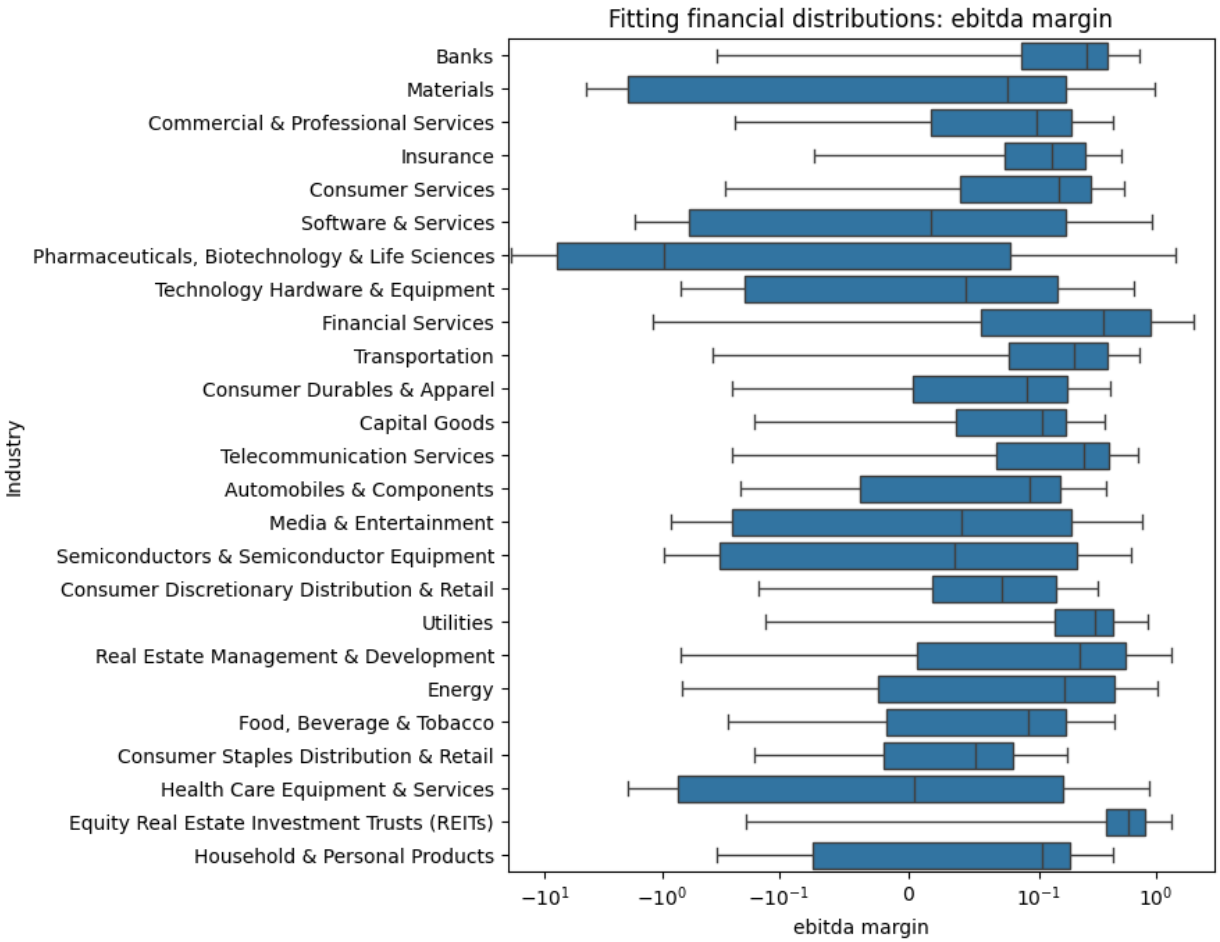


Figure 3: Figure 5. EBITDA margin across unique fitting companies. The missingness table is needed alongside this distribution.

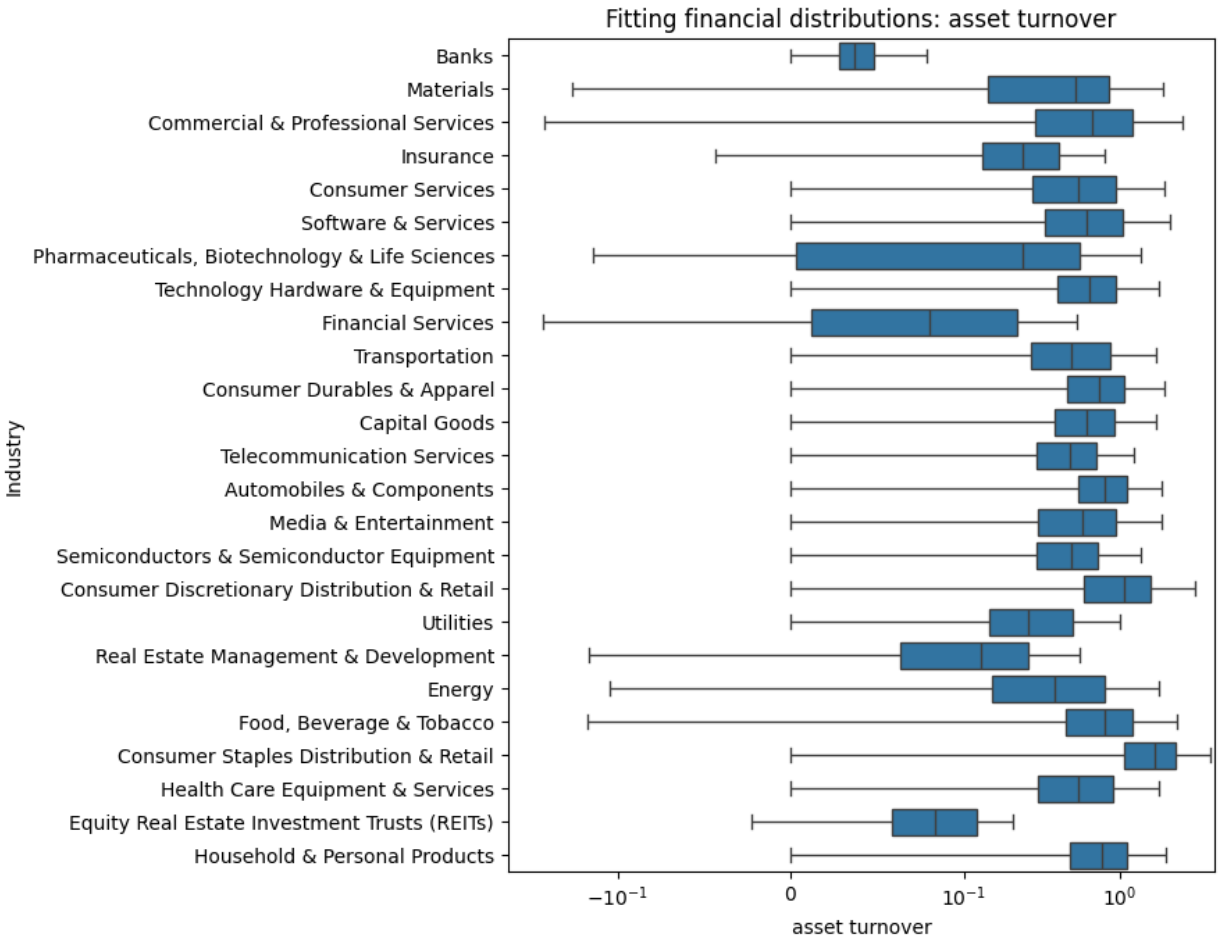


Figure 4: Figure 6. Asset turnover across unique fitting companies. Typical turnover differs substantially between some industries.

Code block 10: Text votes and financial compatibility

Financial compatibility, step by step

There are 75 references: 25 fitting industries times three ratios. Each is a sorted vector of available values from unique fitting companies. Their usable sizes range from 66 to 6,193. All are nonempty and exceed 20 in this supplied dataset, so a branch for references smaller than 20 is unnecessary. A different training dataset would require rechecking that empirical fact before reusing the simplification.

For a finite ratio x in the industry's reference of size n, let L count values strictly below x and R count values less than or equal to x. `searchsorted(..., side='left')` and `side='right'` obtain these counts efficiently. Define the empirical mid-distribution position and compatibility by

$$u(x) = \frac{L + R}{2n}, \qquad t(x) = 2\min(u(x), 1 - u(x)).$$

When there are no ties, $L = R$. With ties, the midpoint treats the whole tied group symmetrically. Middle positions have compatibility near one; tail positions near zero. Values below or above all reference observations can have zero compatibility, which is valid. No logarithm or division by u is used, so an arbitrary offset is unnecessary. This differs slightly from the earlier smoothed rank expression and makes the current formula easier to explain.

For example, take reference $[1, 2, 2, 4]$. For $x = 2$, $L = 1$, $R = 3$, $u = 0.5$, and $t = 1$. For $x = 3$, $L = R = 3$, so $u = 0.75$ and $t = 0.5$. For $x = 5$, $u = 1$ and $t = 0$. These are illustrative arithmetic examples, not experimental project scores.

Which industry's distribution is used?

The lookup uses the **predicted** industry name obtained from the ensemble. It never selects a validation company's true industry's financial distribution. The latter would leak the answer into the confidence features. Fitting industry labels are legitimately used to build the reference dictionary; those are supervised fitting data.

Available per-ratio compatibilities are averaged. If a_i ratios are usable, financial compatibility is $\sum_j t_{ij}/a_i$ and availability is $a_i/3$. When none is available, compatibility is a placeholder of 0.5 and availability is zero. A real observed compatibility of 0.5 has positive availability, so the metamodel can distinguish missing evidence from actual middle-valued evidence. `np.divide(..., out=..., where=...)` implements this without dividing by zero.

`reindex(columns=ratios)` provides NaN columns when a new input has no finances. Such observations remain predicted using text. Infinite supplied ratios are excluded by `isfinite`. Missing values do not become zero-margin companies and are not silently treated as observed averages.

Why this is not a p-value

A formal p-value concerns the probability, under a specified null model, of a test statistic at least as extreme as the observed one. It is not the probability that an industry is correct. These empirical two-sided positions are bounded compatibility features, without a formal test's assumptions, finite-sample validity claim, or multiple-testing interpretation. Averaging them does not create a formal combined p-value. The probability of correctness is instead learned and evaluated in the following block.

The five confidence inputs, in order, are maximum ensemble vote, gap to the second vote, LR/k-NN agreement, mean financial compatibility, and financial availability. Sorting the 25 vote entries makes the last two easy to retrieve. The gap measures how decisively the chosen class beats its nearest alternative; agreement measures whether the base classifiers chose the same class, not whether they are statistically independent.

A middle rank is not a probability density. In a bimodal distribution, a value in the gap between modes could have a middle rank despite low local density. This is an explicit limitation of the simple compatibility feature, not a reason to call it a likelihood. Kernel density estimation would

require bandwidth and other modeling choices; no such method is claimed or hidden in this implementation.

Numbered code references

- **[10.01]** `profiles = {}` — Store the fitting-only financial reference samples.
- **[10.02]** `for industry, group in profile_data.groupby('industry'):` — Separate the unique fitting companies by known industry.
- **[10.03]** `for column in ratios:` — Create one reference for each of the three ratios.
- **[10.04]** `profiles[industry, column] = np.sort(group[column].dropna().to_numpy())` — Sort available values so percentile lookups are quick.
- **[10.05]** `def confidence_features(companies, proba_lr, proba_knn):` — Reuse exactly one confidence-feature calculation.
- **[10.06]** `votes = weight * proba_lr + (1 - weight) * proba_knn` — Combine aligned industry probabilities with the tuning-selected weight.
- **[10.07]** `prediction = votes.argmax(1)` — Choose each row's largest ensemble vote.
- **[10.08]** `top = np.sort(votes, axis=1)` — Sort each vote vector to find its largest two entries.
- **[10.09]** `agreement = proba_lr.argmax(1) == proba_knn.argmax(1)` — Record whether LR and k-NN choose the same industry.
- **[10.10]** `names = le.inverse_transform(prediction)` — Find the industry names predicted by the ensemble.
- **[10.11]** `financial = companies.reindex(columns=ratios)` — Add absent financial columns as missing rather than inventing values.
- **[10.12]** `total, available = np.zeros(len(companies)), np.zeros(len(companies))` — Start each company's compatibility total and evidence count at zero.
- **[10.13]** `for (industry, column), reference in profiles.items():` — Visit each industry-ratio reference once.
- **[10.14]** `values = financial[column].to_numpy()` — Read this ratio in the input-row order.
- **[10.15]** `mask = (names == industry) & np.isfinite(values)` — Use finite values for the predicted industry, never the true industry.
- **[10.16]** `left = np.searchsorted(reference, values[mask], side='left')` — Count reference values strictly below the observation.
- **[10.17]** `right = np.searchsorted(reference, values[mask], side='right')` — Count reference values below or equal to the observation.
- **[10.18]** `percentile = (left + right) / (2 * len(reference))` — Use the midpoint of the two empirical cumulative fractions.
- **[10.19]** `total[mask] += 2 * np.minimum(percentile, 1 - percentile)` — Add a bounded middle-versus-tail compatibility score.
- **[10.20]** `available[mask] += 1` — Count this observed financial contribution.
- **[10.21]** `compatibility = np.divide(total, available, out=np.full(len(companies), 0.5), where=available > 0)` — Average available scores; distinguish no evidence with availability zero.
- **[10.22]** `return prediction, np.column_stack([top[:, -1], top[:, -1] - top[:, -2],` — Return the class codes and five confidence inputs in a fixed order.

Code block 11: Learning confidence

A second prediction problem

The original target has 25 industries; this target has two values: one when the fixed ensemble's industry is correct and zero otherwise. These are different learning problems. The industry LR uses 768 inputs and 25,681 fitting records. The correctness LR uses five inputs and 1,427 calibration records. Calling both models logistic regression does not mean they are accidentally trained twice on the same task.

The calibration subset contains 1145 correct ensemble predictions and 282 errors. Both target classes are present. The default binary logistic model estimates

$$c = \sigma(z) = \frac{1}{1 + e^{-z}}, \qquad z = \beta_0 + \sum_{j=1}^{5} \beta_j h_j.$$

For this fit the intercept is -1.570516 and the five coefficients are listed in the worked-calculation chapter. The positive-class probability is `predict_proba(... )[:, 1]` because the classes are `[0,1]`. The input features are bounded between zero and one, so another scaling layer is not needed merely because it is conventional. Default L2 regularization is retained; the binary model is small and uses 25 optimizer iterations.

The text-only calibrator uses the first three inputs. The financial one includes the remaining two. This small comparison is an ablation: it isolates what financial inputs add while keeping predicted labels identical. Removing the comparison would save one fit but lose the evidence needed to evaluate the financial contribution requested in the brief. It does not involve a large ensemble of confidence models or expensive cross-validation.

The learned compatibility coefficient is slightly negative (-0.028865), while availability has a positive coefficient (0.265546). This is not a reason to reverse a sign manually. Coefficients describe a conditional relationship in this calibration sample, with correlated inputs and regularization; they are not causal economic laws. Overlapping industry ratios make a large positive compatibility effect far from guaranteed. Maximum vote and vote margin have much larger coefficients.

Calling the metamodel a calibrator describes its intended role. It is not proof of perfect calibration. The reliability table and held-out Brier score check whether its confidence matches observed correctness. Distribution shift, missingness changes, and unfamiliar industries can still cause incorrect or overconfident outputs.

Numbered code references

- **[11.01]** `pred_cal, features_cal = confidence_features(df_cal, lr_cal, knn_cal)` — Build confidence inputs from held-out calibration companies.
- **[11.02]** `correct_cal = (pred_cal == y_cal).astype(int)` — Set the binary learning target to whether the ensemble was correct.
- **[11.03]** `text_calibrator = LogisticRegression().fit(features_cal[:, :3], correct_cal)` — Fit the text-only confidence comparison using the first three inputs.
- **[11.04]** `finance_calibrator = LogisticRegression().fit(features_cal, correct_cal)` — Fit correctness confidence with the two additional financial inputs.
- **[11.05]** `pred_val, features_val = confidence_features(df_val, lr_val, knn_val)` — Build the same inputs for validation without fitting anything.

Code block 12: Evaluating confidence

The comparison that can support a financial claim

All three rows use the same ensemble class predictions. Consequently, their weighted F1 is identical at 78.09%. Only their confidence vectors differ. Raw maximum-vote confidence has Brier 0.13738 and official score 67.36%. Text calibration has Brier 0.13592 and score 67.47%. Financial calibration has Brier 0.13558 and score 67.50%.

The financial improvement relative to text calibration is approximately **0.03 percentage points**, not three percent. Their Brier difference is 0.0003410. These are tiny differences on one reused holdout, so they do not establish a robust financial advantage. Reporting the weaker comparison rows is essential; selecting only the flattering number would exaggerate the result.

As a descriptive uncertainty check outside the submitted implementation, 1,000 paired bootstrap resamples of the validation rows give a 95% percentile interval of **[-0.0000850, 0.0007975]** for the reduction in mean Brier. It includes zero. A bootstrap sample draws 3,171 row positions with replacement, so some rows appear more than once and others are omitted. For each draw the same positions select both confidence methods, and the mean of their per-row squared-loss differences is computed. The interval uses the 2.5th and 97.5th percentiles of these 1,000 means; the seed was 42. This estimates uncertainty conditional on the fitted models and an independent-row sampling approximation; repeated representations can violate that approximation, and it does not include variation from refitting the whole pipeline. It is not a private-test guarantee or a formal statement that finance never helps.

The financial version remains the specified inference candidate because the method is intended to combine text and finance. It is not chosen by repeatedly searching validation outcomes. Financial compatibility is retained as a small, interpretable signal and an explicitly evaluated part of the requested analysis, not advertised as a major source of performance.

Numbered code references

- **[12.01]** `confidences = {` — Compare raw votes, text confidence, and financial confidence.
- **[12.02]** `rows = []` — Collect all three approaches using the same predictions.
- **[12.03]** `for name, confidence in confidences.items():` — Evaluate each confidence vector without changing its labels.
- **[12.04]** `rows.append([name, f1_score(y_val, pred_val, average='weighted'),` — Calculate weighted F1, correctness Brier, and the official score.
- **[12.05]** `bonus = pd.DataFrame(rows, columns=['Method', 'Weighted F1', 'Brier', 'Official score']).set_index('Method')` — Give the confidence comparison clear column names.
- **[12.06]** `display(bonus)` — Display the complete confidence comparison, including weaker results.

Actual confidence results

Method	Weighted F1	Brier	Official score
Raw ensemble	0.780881	0.137380	0.673604
Text calibration	0.780881	0.135923	0.674742
Financial calibration	0.780881	0.135582	0.675008

Code block 13: Confidence and human review

Calibration diagrams

If confidence is well calibrated, a large group with confidence around 0.8 should be correct about 80% of the time. A single observation cannot demonstrate calibration. The code creates 11 equally spaced boundaries from zero to one, which define 10 intervals. `include_lowest=True` includes zero. `observed=True` excludes empty bins from the displayed summary, explaining why there can be fewer than ten plotted points.

Within each occupied bin, the table reports the number of observations, mean confidence, and accuracy. The diagram plots mean confidence on the horizontal axis and accuracy on the vertical axis. Points below the diagonal are overconfident; points above are underconfident. The diagonal is $y = x$, not the line $y = 1$. The latter would mean perfect accuracy regardless of reported confidence.

In the 0.8–0.9 bin, observed accuracy is below its mean confidence. The highest bin has many more records and is close to the diagonal. The lowest occupied bin has only 11 records and is noisy. Visual closeness to the diagonal is useful descriptive evidence, but does not prove exact conditional calibration for every industry or subgroup.

Review policy and its denominator

Confidence strictly below 0.8 is flagged for human review. Exactly 0.8 is unflagged. The threshold is an explicit starting policy, not an official requirement or a validation-optimized optimum. A higher threshold increases review workload and generally catches more errors; it also sends more correct predictions for review. The operational costs of review and error were not provided, so there is no basis for claiming 0.8 uniquely optimal.

The policy flags 1,169 of 3,171 records (36.9%) and captures 496 of 678 errors (73.2%). Flagged accuracy is 57.57%, versus 90.91% for unflagged records. Thus the policy identifies a more difficult subset, but still misses 182 errors. No actual human review outcome is simulated. All records retain their original predictions in the official score.

The unflagged accuracy above 90% does **not** meet an 85% official-score target. It concerns a selected subset and a different metric. Discarding low-confidence rows would change the evaluation denominator and would not follow the stated task.

Numbered code references

- **[13.01]** `confidence_val = confidences['Financial calibration']` — Use the financial-calibrated confidence for inference and review.
- **[13.02]** `outcomes = pd.DataFrame({'Confidence': confidence_val, 'Correct': pred_val == y_val})` — Pair confidence with observed correctness for evaluation only.
- **[13.03]** `outcomes['Bin'] = pd.cut(confidence_val, np.linspace(0, 1, 11), include_lowest=True)` — Divide zero to one into ten confidence intervals.
- **[13.04]** `reliability = outcomes.groupby('Bin', observed=True).agg(` — Compare mean confidence, actual accuracy, and sample size in each bin.
- **[13.05]** `display(reliability)` — Show counts so small confidence bins are not overinterpreted.
- **[13.06]** `fig, ax = plt.subplots()` — Create the confidence-versus-accuracy plot.
- **[13.07]** `ax.plot([0, 1], [0, 1], '--', color='gray', label='Perfect calibration')` — Draw the ideal calibration line, where confidence equals accuracy.
- **[13.08]** `ax.plot(reliability.Mean_confidence, reliability.Accuracy, 'o-', label='Validation bins')` — Plot the observed validation-bin values.
- **[13.09]** `ax.set(xlabel='Mean predicted confidence', ylabel='Observed accuracy', title='Confidence versus validation ac...` — Label the calibration axes and title.
- **[13.10]** `ax.legend()` — Identify the observed and ideal lines.
- **[13.11]** `fig.tight_layout()` — Fit the plot labels within the figure.

- **[13.12]** `plt.show()` — Display the calibration plot.
- **[13.13]** `review = confidence_val < 0.8` — Flag confidence strictly below the fixed 0.8 review threshold.
- **[13.14]** `display(outcomes.groupby(review).agg(Count=('Correct', 'size'), Accuracy=('Correct', 'mean')))` — Compare flagged and unflagged records; True means human review.
- **[13.15]** `print('Review fraction:', review.mean(), '| errors captured:',` — Show review workload and the number of errors the flags capture.

Occupied confidence bins

Bin	Records	Mean confidence	Accuracy
(0.2, 0.3]	11	0.2807	0.3636
(0.3, 0.4]	128	0.3644	0.4375
(0.4, 0.5]	258	0.4496	0.4496
(0.5, 0.6]	235	0.5484	0.5574
(0.6, 0.7]	237	0.6493	0.6287
(0.7, 0.8]	300	0.7524	0.7233
(0.8, 0.9]	617	0.8579	0.8201
(0.9, 1.0]	1385	0.9375	0.9487

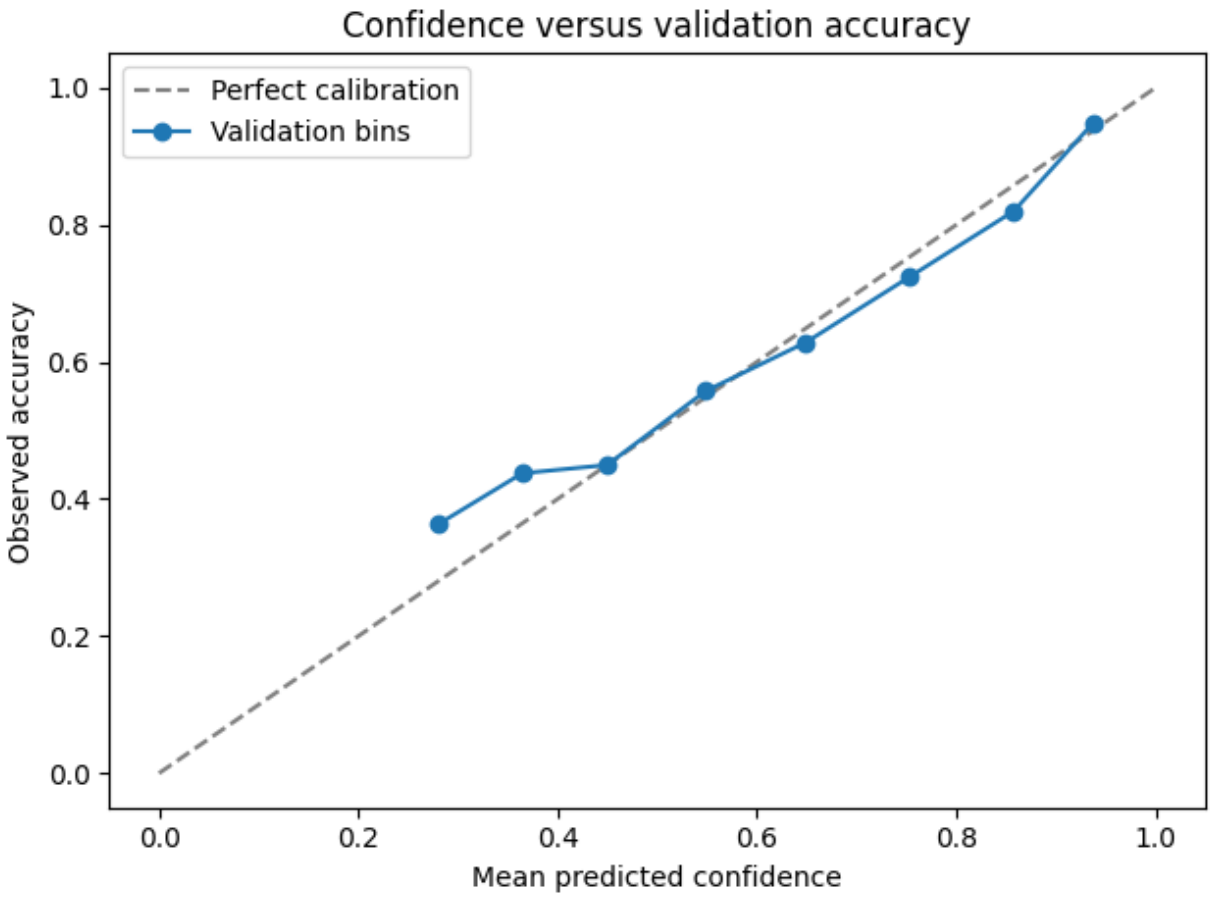


Figure 5: Figure 7. Validation reliability diagram. The sample sizes behind these points appear in the table; near-diagonal points do not prove calibration in every subgroup.

Code block 14: Predicting new companies and checking repeated representations

The prediction contract

`predict_industry(companies)` accepts a DataFrame with the supplied string embedding column. The three financial columns are optional; absent columns become missing evidence. IDs are useful for joining finances and displaying results but are not themselves features in the prediction function. An input may accidentally contain an industry column, but the function does not read it.

The output is a tuple `(industry_names, confidence)`. Both are one-dimensional vectors of length equal to the number of input rows, in that same order. Names come from the fitting label encoder; confidence comes from the financial correctness model. The function does not return review flags because the official interface requests two vectors; a caller can calculate `confidence < 0.8` separately. When scoring these outputs, truth and predictions must use the same label representation: compare names with the input industry's names, or encode the returned names with `le.transform` before comparing with the integer-coded `y_val`.

New rows are parsed, projected using the fitted PCA, scored by the fixed classifiers, combined using the selected voting weight, and passed through the exact same confidence-feature function used during calibration. There is no refitting and no lookup of a new record's true industry. Separately supplied financial tables should be aggregated and joined by ID using the loading-cell procedure before calling this function.

The public demonstration removes the industry column before predicting. Verification also changed true labels deliberately, reversed row order, removed finances entirely, and replaced ratios with infinity. The predictions remained target-independent and order-consistent, and confidence stayed finite and bounded. A real-data execution is stronger evidence than syntax checking alone.

Repeated representations and limits

There are 4,578 repeated stored embedding strings in the full main table, including 17 representation groups with conflicting industry labels. Distinct IDs can share a representation even when their original descriptions are unavailable for inspection. ID grouping therefore prevents the known same-company overlap, but does not guarantee entirely novel descriptions or numeric representations in validation.

Exactly 455 validation records (14.35%) have an identical stored embedding string in the fitting set. Their ensemble accuracy is 99.34%; accuracy on the other 2,716 records is 75.15%. The overall 78.62% accuracy is a mixture of those groups. String equality is the implemented diagnostic; it should not be overstated as a complete audit of every possible numeric near-duplicate or semantic duplicate.

The 85% official-score target remains unmet. Private-test inputs and labels are unavailable. The local holdout has been examined in earlier versions, rare classes have small evaluation supports, and confidence may shift on another population. The cloud runtime is measured, while the Intel Mac runtime is not. None of these limitations can be repaired by changing wording, discarding difficult rows, or presenting subset accuracy as the official score.

Numbered code references

- **[14.01]** `def predict_industry(companies):` — Expose the required label-vector and confidence-vector prediction interface.
- **[14.02]** `X = embeddings(companies)` — Convert new descriptions with the same embedding parser.
- **[14.03]** `prediction, features = confidence_features(companies, lr.predict_proba(X), knn.predict_proba(pca.transform(X)))` — Reuse both classifiers, fitting PCA, and the confidence features.
- **[14.04]** `return le.inverse_transform(prediction), finance_calibrator.predict_proba(features)[:,`

`1]` — Return industry names and correctness confidence in input-row order.

- **[14.05]** `example = df_val.drop(columns='industry').head(3)` — Demonstrate prediction after removing the true industry column.
- **[14.06]** `labels, confidence = predict_industry(example)` — Call the same public prediction function used for new companies.
- **[14.07]** `display(pd.DataFrame({'ID': example.id.to_numpy(), 'Predicted industry': labels, 'Confidence': confidence}))` — Display three real examples with their IDs and predicted outputs.
- **[14.08]** `seen = df_val.business_description_embedding.isin(df_fit.business_description_embedding)` — Check exact stored embedding strings shared with fitting records.
- **[14.09]** `display(pd.DataFrame({'Embedding seen in fitting': seen.to_numpy(), 'Correct': pred_val == y_val})` — Compare accuracy on seen and unseen stored representations.
- **[14.10]** `print('Whole notebook seconds:', round(perf_counter() - started, 1))` — Report elapsed notebook time through the final code cell.

Worked calculations using the actual fitted model

These examples come from the current validation outputs. Positions identify rows within the local validation array, not company IDs. True labels are used here only to explain evaluation; they are not read by the prediction function. Displayed rounding can cause tiny last-digit differences from full-precision calculations.

The learned correctness equation

Quantity	Fitted value
Intercept	-1.5705164242
Maximum ensemble vote coefficient	2.3410667457
Vote margin coefficient	1.9681715941
Model agreement coefficient	0.0533042306
Financial compatibility coefficient	-0.0288652211
Financial availability coefficient	0.2655462457

For a feature vector h, multiply each feature by its coefficient, add the intercept, and apply $1/(1+e^{-z})$. The following cases show each step.

Case A: correct prediction

Validation position **2** has true industry **Consumer Services**. The ensemble predicts **Consumer Services**.

Industry	LR probability	k-NN probability	25/75 vote
Consumer Services	0.88306143	0.87150562	0.87439457
Consumer Discretionary Distribution & Retail	0.00350119	0.12849438	0.09724608
Financial Services	0.03547311	0.00000000	0.00886828

For the selected class, $0.25 \times 0.88306143 + 0.75 \times 0.87150562 = 0.87439457$. Subtracting the second vote, 0.09724608, gives margin 0.77714849. Model agreement is 1. These are [10.06]–[10.09].

The next calculations use the **predicted** industry's financial references. Each row has $u = (L + R)/(2n)$ and $t = 2\,\mathrm{min}(u, 1-u)$.

Ratio	Observed value	n	L / R	u	t
net profit margin	0.01862862	469	211 / 211	0.449893	0.899787
ebitda margin	0.09684027	260	92 / 92	0.353846	0.707692
asset turnover	0.67085282	477	284 / 284	0.595388	0.809224

The mean compatibility is $(0.899787 + 0.707692 + 0.809224)/3 = 0.80556780$. All three ratios are observed, so availability is $3/3 = 1$. See [10.18]–[10.22].

Feature	h	Coefficient times h
Maximum ensemble vote	0.87439457	2.04701606
Vote margin	0.77714849	1.52956159
Model agreement	1.00000000	0.05330423
Financial compatibility	0.80556780	-0.02325289
Financial availability	1.00000000	0.26554625

Adding the five products to the intercept -1.57051642 gives $z = 2.30165881$. Therefore $c = 1/(1 + e^{-2.30165881}) = 0.90901433$. The correctness target is 1, so its Brier contribution is $(0.90901433 - 1)^2 = 0.00827839$.

This correct prediction is unflagged because its confidence exceeds 0.8. It contributes a small, positive confidence error rather than zero: the estimate was not certainty.

Case B: incorrect prediction

Validation position **0** has true industry **Technology Hardware & Equipment**. The ensemble predicts **Capital Goods**.

Industry	LR probability	k-NN probability	25/75 vote
Capital Goods	0.32255329	0.72524570	0.62457260
Technology Hardware & Equipment	0.55181219	0.13901347	0.24221315
Consumer Durables & Apparel	0.05461638	0.13574082	0.11545971

For the selected class, $0.25 \times 0.32255329 + 0.75 \times 0.72524570 = 0.62457260$. Subtracting the second vote, 0.24221315, gives margin 0.38235945. Model agreement is 0. These are [10.06]–[10.09].

The next calculations use the **predicted** industry's financial references. Each row has $u = (L + R)/(2n)$ and $t = 2\,\mathrm{min}(u, 1 - u)$.

Ratio	Observed value	n	L / R	u	t
net profit margin	0.11988204	2276	1876 / 1876	0.824253	0.351494
ebitda margin	0.16179975	727	521 / 521	0.716644	0.566713
asset turnover	1.33823865	2305	2076 / 2076	0.900651	0.198698

The mean compatibility is $(0.351494 + 0.566713 + 0.198698)/3 = 0.37230162$. All three ratios are observed, so availability is $3/3 = 1$. See [10.18]–[10.22].

Feature	h	Coefficient times h
Maximum ensemble vote	0.62457260	1.46216614
Vote margin	0.38235945	0.75254900
Model agreement	0.00000000	0.00000000
Financial compatibility	0.37230162	-0.01074657
Financial availability	1.00000000	0.26554625

Adding the five products to the intercept -1.57051642 gives $z = 0.89899840$. Therefore $c = 1/(1 + e^{-0.89899840}) = 0.71074363$. The correctness target is 0, so its Brier contribution is $(0.71074363 - 0)^2 = 0.50515651$.

LR alone selected the true Technology Hardware & Equipment class, but the larger k-NN vote led the ensemble to Capital Goods. This illustrates that an ensemble can worsen one observation while improving aggregate performance. Calibration raises this case's confidence above its raw maximum vote, increasing its individual loss, yet still improves the whole-set mean loss. The case is flagged because confidence is below 0.8.

A class-level F1 calculation: Banks under LR

The raw matrix has TP=103, FP=3, and FN=3. There are 106 true Banks records. Thus precision and recall are both $103/106 = 0.97169811$, and

$$F_1 = \frac{2 \times 103}{2 \times 103 + 3 + 3} = 0.97169811.$$

Its contribution to weighted F1 is $(106/3171) \times 0.97169811 = 0.03248187$. Its contribution to macro F1 is $0.97169811/25 = 0.03886792$. Neither contribution is the complete model score; the other 24 industries must also be included.

The complete official score

The ensemble weighted F1 is 0.780881472 and its mean correctness Brier is 0.135581702. Therefore $S = 0.780881472(1 - 0.135581702) = 0.675008233$ or **67.50%**. A fraction such as 0.6750 becomes 67.50% by multiplication by 100. This score uses all 3,171 records.

Oral-exam questions and model answers

These are practice answers to likely questions, not scripts to memorize without understanding. Start with a short answer, point to the relevant numbered code, and then explain a calculation or limitation. The professor can ask variations; the principles below should help you reason through them.

Project and data

1. What problem does the project solve?

It assigns one of 25 industry labels to a company record using its supplied business-description embedding. It also estimates the probability that the selected label is correct. The mandatory comparison uses logistic regression and k-NN; the optional system soft-votes their probabilities and uses financial evidence in the correctness-confidence model. References: [06], [10], [14].

2. How many companies do you have?

There are 31,706 main records and 31,705 unique IDs. One company has two different embedding records. The uploaded subset is smaller than the brief's statement about more than 40,000 companies; I report the file actually analyzed rather than adding financial-table rows to make the count larger.

3. What is an embedding? Can you interpret coordinate 27?

It is a numerical representation of text produced by a pretrained model. Nearby representations often correspond to related meanings, but a particular coordinate generally has no simple industry label. I use all 768 provided coordinates rather than claiming that coordinate 27 measures a known business activity. Reference: [03.10].

4. Did you train the language model?

No. The course already supplies the embeddings. There is no text-model download, embedding fine-tuning, GPU, or neural-network training in this notebook. This keeps the project within the intended libraries and resource limits.

5. Why is the ID not included among the features?

It is a join/group identifier, not a meaningful continuous measurement of a business. It may contain accidental ordering or source-system patterns. I use it to connect finances and keep company records together, not to obtain a potentially misleading classification shortcut. References: [02.06], [03.03].

6. Why not delete the repeated ID?

The two records have different embeddings and the same industry, so they are not identical main rows. Keeping them together preserves potentially useful description information. For financial reference distributions, which describe unique companies, I count that company once. References: [03], [09.01].

7. What if two financial rows for one ID disagree?

I take the median separately for each finite numerical ratio. This is an explicit within-company summary rather than arbitrarily taking the first row. The data do not provide enough period information to claim that this is a specific year's financial statement. Reference: [02.05].

8. Does a missing value mean a zero margin?

No. It means the observation is unavailable. A true zero is a valid number. Treating missing values as observed zero would manufacture financial evidence; this method instead averages only available ratio compatibilities and supplies a separate availability feature. References: [10.15], [10.20]–[10.22].

9. Why retain negative margins and extreme finite observations?

Negative margins can represent real losses. An extreme ratio can reflect a small denominator or a genuine exceptional company. Empirical ranks limit the influence of magnitudes without declaring every unusual observation invalid. Infinity is different: it is not a finite numerical observation and is marked missing. References: [02.05], [09], [10.18].

10. What does `validate='many_to_one'` protect against?

It checks that each main row matches at most one aggregated financial row. Otherwise a join on repeated keys could multiply records, altering sample counts and weighting companies several times. A left join deliberately keeps main rows with no match. Reference: [02.06].

Splitting and leakage

11. Why four sets instead of only train/test?

They perform different jobs: fit the industry models, choose the voting weight, learn correctness confidence, and evaluate the completed procedure. Sharing a set between tuning and confidence learning would allow model choices to depend on the same correctness outcomes used to teach confidence. Reference: [03].

12. Where do 81%, 4.5%, 4.5%, and 10% come from?

Reserve 10% of IDs. Of the remaining 90%, keep 90% for fitting: $0.9 \times 0.9 = 0.81$. The other 9% is split equally: $0.9 \times 0.1 \times 0.5 = 0.045$ each. Group counts are rounded, and one ID has two rows, so actual row counts are used in outputs.

13. What is data leakage in this project?

It would include fitting PCA on all companies, fitting an industry profile using validation targets, using a company's true validation industry to choose a financial reference, or separating records of the same company across fitting and validation. The code prevents these known paths. Repeated representations across distinct IDs remain a disclosed limitation. References: [03], [05.03], [10.15], [14.08].

14. Why can financial duplicate aggregation happen before the split?

It uses only records of the same company and no labels. Company grouping then keeps that ID together. In contrast, industry-level financial profiles combine information across companies using their known labels, so they must be fitted only on fitting companies. References: [02.05], [09.01], [10.02].

15. Does parsing a validation embedding count as fitting on it?

No. Parsing is a deterministic conversion of that row's stored values. Learning a mean, PCA direction, model coefficient, or industry reference from evaluation records would be different. The important distinction is conversion versus estimation from a population. References: [03.10], [05.03].

16. Is the split stratified?

No. It groups by company ID and uses a fixed seed. Industry proportions can vary. All held-out industries do occur in fitting data in this run, and the assertion explicitly checks that requirement. I do not call this a stratified split. References: [03.02], [04.02].

17. Why not refit everything on all supplied records after evaluation?

A refitted classifier would produce different predictions and correctness patterns from those used to teach the current calibrator. A defensible full-data procedure would need new held-out or out-

of-fold confidence training and its own evaluation. The current frozen fits keep the learning roles simple and honest. Refitting on validation and reusing its old score would not validate the new model.

18. Is your final validation completely untouched?

It was reserved from fitting and tuning in the procedure, but its results were inspected in earlier project versions. I therefore describe current figures as reused-holdout results, not a fresh independent confirmation. The professor's private labels have never been available.

19. Why is seed 42 used? Does that number have statistical meaning?

It makes the random split repeatable and matches the course's example seed. Its numeric value has no special statistical power. Another fixed seed would define another valid split with somewhat different results; choosing a flattering seed after seeing validation would be another form of selection.

20. Do fixed seeds guarantee identical outputs on every machine?

They control the seeded random operations. Different package versions, numerical libraries, thread behavior, and floating-point rounding can still change last digits or close decisions. Reproduction therefore also records data, versions, settings, and the actual evaluation procedure.

Labels, PCA, and model choices

21. Why use LabelEncoder instead of one-hot targets?

These scikit-learn classifiers accept a single categorical target per row. Integer codes provide that representation without implying a meaningful ordering. One-hot encoding would add an unnecessary target matrix here. Reference: [04].

22. Why fit the label mapping on fitting data only?

It defines the model's known classes using fitting information and is reused consistently. The subset check verifies the held-out labels are representable. Independently fitted mappings could assign incompatible codes, so probability column 3 would not necessarily mean the same industry. References: [04.01]–[04.03].

23. Why do you show class frequencies?

They reveal imbalance: Materials supplies about 26.5% of fitting records while the smallest class has only 151. This affects the available learning evidence and the interpretation of weighted versus macro results. The skeleton permits a table, so an extra chart is unnecessary. References: [04.04]–[04.05].

24. What exactly does PCA learn?

It learns the fitting mean and orthogonal directions of maximum feature variance. New rows are centred with that same mean and multiplied by the learned component directions. It does not learn industry labels, although labels color the scatterplot afterward. References: [05.02]–[05.04].

25. Is 87.6% explained variance an accuracy of 87.6%?

No. It concerns variance of embedding coordinates, not correct classification. Important discriminative information can occur in low-variance directions. Actual accuracy and F1 must be measured separately using industry targets. References: [05.12], [07.03].

26. Why can industries overlap in the plot yet still be classified?

The plot contains only two coordinates, retaining about 25.3% of feature variance. The classifiers use 128 PCA coordinates or all 768 original coordinates. Distinctions absent from the compressed picture may remain in the dimensions not displayed.

27. Why 128 PCA components?

It was a useful development choice for k-NN, balancing information retention and distance cost. The bounded checks included 64 components and raw coordinates. This does not establish a universal optimum or prove that all removed variation was noise.

28. Why not standardize or whiten every coordinate?

The embedding coordinates already come from one numerical representation rather than

unrelated physical units. Rescaling changes distance geometry and need not help. The tested whitening alternatives weakened the ensemble, so another transformation would add work without measured benefit.

29. Does using different feature dimensions make the model comparison invalid?

It compares two practical pipelines on the same companies and labels. It is valid for selecting the better implemented pipeline, but it cannot isolate an intrinsic algorithm-only advantage of LR over k-NN. A controlled algorithm study would hold more preprocessing choices fixed.

30. Why is logistic regression a classifier?

Its learned linear scores are transformed into categorical probabilities with softmax; the predicted class is the largest probability. The name does not mean that industry codes are treated as continuous numerical measurements. Reference: [06.01].

31. What does C=10 mean?

It is the inverse regularization-strength parameter. A larger C weakens the L2 penalty and allows larger weights. Development comparisons informed this choice; larger C values did not keep improving performance. It is not a probability, a neighbor count, or an iteration count.

32. Why C=10 if C=1 was stronger for LR alone?

The retained final classifier is a combination. A model setting can complement the other model better even when its standalone score is slightly weaker. The development blend was stronger with C=10; this does not justify claiming that C=10 is universally superior.

33. What does max_iter=1000 do?

It caps optimization iterations. The measured fit stopped after 188 iterations, so the code did not force exactly a thousand passes. Convergence and runtime are checked rather than inferred from that cap.

34. What does k-NN learn when `.fit` is called?

It retains fitting feature vectors and target information for neighbor queries. Unlike LR it does not optimize a matrix of industry weights. Most prediction work is finding and combining neighbors for a new row. Reference: [06.05].

35. Why fifteen neighbors and distance weights?

The tested 5/15/25-neighbor choices favored 15 in the retained representation. Distance weighting gives closer neighbors more influence rather than treating the fifteenth as identical evidence to the closest. Both are development choices, not theorem-derived constants.

36. What happens if a query has an identical fitting embedding?

There can be zero-distance neighbors. scikit-learn handles them specially, assigning votes only among the zero-distance matches rather than literally calculating infinity as a weight. Conflicting labels among identical representations can still make a mistake unavoidable for a representation-only classifier.

37. Why not add the nonlinear classifier you tested?

The RBF check at C=10, gamma=4 had tuning F1 around 0.773 and required about 326 seconds just for fitting. It was weaker than the simple voting ensemble and far more expensive. Additional code and oral-exam complexity were not justified by a gain. The notebook therefore does not import or fit that classifier.

Evaluation and the official target

38. What is the difference between precision and recall?

Precision asks how often a predicted industry is correct; recall asks how many of that industry's actual records were found. A model can increase one while decreasing the other. F1 combines them through a harmonic mean, penalizing an imbalance between the two. Reference: [07.04].

39. Why report both weighted and macro F1?

Weighted F1 follows the class-support weighting in the official scorer. Macro F1 exposes how performance changes when each industry has equal influence. The lower macro values show that strong common-class results conceal weaker performance on some smaller industries.

40. Is weighted F1 always the same as accuracy?

No. Accuracy counts total correct records. Weighted F1 first calculates each class's precision/recall combination and then averages by support. Micro F1 equals accuracy in this single-label multiclass setting; weighted F1 need not. Reference: [07.03].

41. Which classifier is best?

LR is the stronger required individual classifier on this holdout: weighted F1 77.78% versus 77.14%. The combined predictions reach 78.09%. The performance conclusion is conditional on this data and procedure, not an assertion that LR always wins.

42. Are large classes always easy?

No. The size/error correlations are negative but imperfect. Commercial & Professional Services remains the weakest class despite hundreds of fitting examples. Feature ambiguity, label definitions, overlapping activities, and information not captured by embeddings can also matter. Reference: [07.05].

43. Is the diagonal of your normalized confusion plot precision?

No: rows are true industries, so row normalization makes the diagonal recall. The raw count matrix and the classification report permit precision to be calculated using prediction-column totals. Reference: [07.09].

44. Why remove the diagonal before listing common confusion pairs?

Correct predictions are generally the largest counts and would hide the important errors. The diagonal is removed only for the subsequent ranking, after the separate normalized heatmap has been constructed. The metric calculations remain unchanged. Reference: [07.13].

45. Do the two models make independent errors?

No independence assumption is made. They share representations and fitting labels, and both fail on 520 records. Their 173 and 187 unique errors provide some useful diversity, but the heavy overlap limits the ensemble improvement. Reference: [07.17].

46. What is a percentage point?

A change from 67.47% to 67.50% is 0.03 percentage points. It is not a three-percent increase. The absolute fraction changes by about 0.0003, while a relative percentage change would divide that change by the original score.

47. Have you achieved the 85% target?

No. The measured official score is 67.50%. The current 78.09% weighted F1 is itself below 85%, and the official score cannot exceed it. Private-test performance remains unknown. Reference: [08.10].

48. If the high-confidence group's accuracy exceeds 90%, is that enough?

No. It is accuracy on a selected subset, while the official score uses every prediction, support-weighted F1, and a Brier penalty. Removing low-confidence records would change the task and hide difficult cases. References: [08], [13].

49. Could you improve the reported result just by discarding uncertain companies?

Not while following this evaluation. Every company must receive a label and confidence. Review flags indicate operational uncertainty but do not remove any row from F1 or Brier. Any selective result must state its denominator and cannot replace the full-set score.

50. Why is a private score not guaranteed by this holdout?

The private companies and labels are unavailable. Their class mix, missingness, and representation novelty may differ. Sampling variation and reuse of the local holdout also limit generalization claims. Only an actual private evaluation can establish its score.

Financial evidence and confidence

51. Why do finances adjust confidence instead of changing industry predictions?

The simpler text models already provide the required labels. Financial ratios overlap and have substantial missingness; their interpretation is useful as supporting evidence. A bounded

financial-label classifier did not justify a more complex prediction pipeline. The chosen approach isolates the financial contribution through confidence comparison.

52. Why use each predicted industry's profile rather than the true profile?

The true industry is unavailable for a new company. Selecting a validation profile using its target would leak the answer. The profile must be selected from the model's current prediction, with the correctness model learning whether such evidence is useful. Reference: [10.15].

53. Why use medians for duplicate rows but percentiles for confidence?

They solve different problems. A within-company median creates one financial value for a join. An industry reference percentile describes where that value lies among fitting companies in a predicted industry. Neither is a substitute for a learned correctness probability.

54. Is the compatibility value a p-value or a probability that the label is correct?

No. It is an empirical middle-versus-tail score without a formal hypothesis-test validity claim. Averaging three such scores does not create a combined p-value. The binary logistic model supplies a separate estimated correctness probability. References: [10.18]–[10.22], [11].

55. Why no Gaussian or log-normal assumption?

Ranks require ordering finite observations rather than fitting a parametric distribution. The data include negative values, heavy tails, and industry-dependent missingness. A normality test or forced log transformation would add assumptions and code without being needed for the chosen method.

56. What exactly do left and right insertion positions mean?

Left counts values strictly below the observation and right counts values below or equal to it. Their average divided by the reference size gives the mid-distribution position and treats tied values symmetrically. References: [10.16]–[10.18].

57. Why remove the old small-sample branch?

Every one of the 75 actual financial references has at least 66 usable observations. The condition never affected this supplied dataset. Removing it simplifies the actual task rather than adding a generalized engine for an unobserved scenario. A different training dataset would need a fresh check.

58. Why remove the rank offset? Can compatibility be zero?

Zero is allowed because this score is a bounded input feature, not a quantity whose logarithm or reciprocal is taken. A value beyond the entire reference sample can legitimately have zero middle compatibility. The correctness model still returns a finite sigmoid confidence; no artificial offset is necessary.

59. Does an unusual financial value prove the industry is wrong?

No. Exceptional companies can be correctly classified, and typical financials can appear in another industry. Compatibility is only supporting evidence. In fact, its small fitted coefficient is slightly negative conditional on the other confidence features, so the data do not support a strong monotonic story.

60. What if all three ratios are absent?

Compatibility is the placeholder 0.5 and availability is zero. With observed ratios, an average compatibility of 0.5 has positive availability, making these situations distinguishable. Prediction still works using text. References: [10.11], [10.21]–[10.22].

61. Why use a separate availability feature?

The confidence model should know whether a compatibility value comes from three observations, one observation, or none. Otherwise the placeholder for missing evidence could be mistaken for an observed score. Missingness can itself correlate with correctness, but its coefficient is not a causal economic finding.

62. Why train another logistic regression?

It predicts a different target: whether the ensemble label is correct. It uses 1,427 held-out calibration records and five bounded confidence inputs, rather than fitting 25 industry categories from 768 coordinates. This small model is a simple learned probability adjustment. Reference:

[11].

63. Why keep the text-only correctness model?

It provides the controlled comparison needed to assess added financial evidence. The two calibrators have identical labels and base predictions; their difference is whether the last two inputs are included. The extra fit is small, and without it I could not attribute a Brier change to financial information even descriptively.

64. Why is the financial compatibility coefficient negative?

The fitted value is -0.028865, small compared with the maximum-vote and margin coefficients. It is conditional on other correlated inputs and regularization, not a universal direction of financial truth. I report it rather than change its sign to fit an expected story. The tiny aggregate financial benefit is not strong evidence of general predictive power.

65. Is the financial improvement statistically established?

No. The observed Brier reduction is 0.0003410. An approximate paired bootstrap interval includes zero and has sampling assumptions, especially questionable with repeated representations. This single reused holdout does not establish a robust or causal advantage. Reference: [12].

66. Is correctness confidence a confidence interval?

No. It is a point estimate of a Bernoulli event probability. A confidence interval would quantify uncertainty about a parameter or performance estimate under a specified sampling procedure. These are different concepts despite the shared word confidence.

67. Is calibration the same as being accurate?

No. A system can be well calibrated but often wrong, for example honestly reporting a low success probability. Calibration checks whether confidence matches event frequency; accuracy checks how often the chosen labels are right. In this notebook calibration never changes the predicted labels or F1.

68. Does calibration always lower confidence?

No. It can raise or lower it based on observed correctness patterns. The worked incorrect case is assigned confidence about 0.711 rather than raw maximum vote 0.625; its individual Brier loss increases even though mean Brier improves. A probability model is not an oracle for each observation.

69. Why not report confidence 1 for every prediction?

Then each error contributes the maximum loss of one, while correct cases contribute zero. The Brier term becomes the error rate, and the official score becomes weighted F1 times accuracy. This is lower than the retained learned confidence result. Reference: [08].

70. Why not report confidence 0 or raise every confidence to the sixth power?

Always zero penalizes every correct prediction. An exponent may help some errors while harming correct cases and has no general justification for this year's formula. The supplied scoring rule is minimized in expectation by the actual conditional correctness probability, which the metamodel tries to estimate.

Review, software, and reproduction

71. How do you read the calibration graph?

Horizontal position is mean predicted confidence within a bin; vertical position is observed accuracy in that bin. Below the diagonal means overconfidence and above means underconfidence. Counts matter: a bin with 11 cases is far less stable than one with more than a thousand. Reference: [13].

72. Why are there fewer than ten points?

The code defines ten intervals but displays only occupied groups. Empty intervals contain no measured mean or accuracy. `observed=True` avoids presenting empty groups as real observations. Reference: [13.04].

73. Why 0.8 for human review?

It is an explicit starting policy, not a professor-mandated or optimized number. Choosing an

optimal threshold would require review and error costs or a separate policy-selection procedure. I report its workload and captured errors without claiming uniqueness. Reference: [13.13].

74. How many errors does review miss?

It catches 496 of 678 errors, so 182 remain among unflagged records. The review fraction is 36.9%. High confidence therefore does not eliminate mistakes, and review flags do not simulate corrected human labels. Reference: [13.15].

75. Why does the prediction function return a tuple?

That is the optional deliverable's interface: one vector of labels and one of confidence values. Each must have one entry per input row. Review flags or extra diagnostic tables can be computed separately without changing that interface. Reference: [14.04].

76. How did you check there is no direct target use in prediction?

The demonstration drops the industry column. Additional checks supplied deliberately wrong target strings and obtained the same outputs. The function reads embeddings and financial ratios, while the learned models and reference tables were constructed from fitting data. This tests the prediction path, not a claim that all possible leakage has been disproved.

77. Why use the same feature function for calibration and new predictions?

It prevents training-serving mismatch: column order, missing-value handling, and percentile definitions must be identical. Repeating that logic manually in multiple cells would be longer and easier to get wrong. The helper is a real reuse, not an unnecessary architecture. References: [10.05], [11.01], [14.03].

78. Why not serialize models or add a Python package?

The deliverable is a notebook that trains and predicts. A deployment package, model registry, persistence format, and application server are not required. A fresh-kernel run is enough to reproduce this analysis; extra infrastructure would make it harder to understand without solving an assignment need.

79. Is the whole program limited to exactly two threads?

No. Numerical-library limits and k-NN parallel jobs control different layers. They are conservative settings, but they do not establish an exact global thread-count guarantee. Runtime and memory are measured independently. References: [01.15], [06.02].

80. Have you proved it runs within 45 minutes on the Intel Mac?

No direct Mac benchmark was performed. The measured cloud run took 221.1 seconds including checks, with fitting-related work taking 44.3 seconds. This suggests a lightweight implementation, but the laptop still needs its own fresh run before claiming that hardware-specific requirement is verified.

81. Why is measured memory much larger than the feature matrix?

The process also holds the original embedding strings, DataFrames, parsed temporary lists, multiple feature sets, PCA scores, models, and plotting objects. A float32 fitting matrix's byte count is only one component. The reported runner-plus-kernel peak upper bound is 1.37 GiB, not a theoretical count of a single array.

82. Why pin software versions?

Defaults and numerical behavior can change, including PCA solver selection and array/view handling. The tested Python and package versions make reproduction more reliable. Seeds help with random choices but do not replace version and input provenance.

83. Does 99.34% accuracy on repeated representations prove excellent generalization?

It describes a favorable subset whose stored embedding string is already seen in fitting. Accuracy on unseen stored representations is 75.15%. Overall performance mixes those groups. Neither figure can be interpreted as a guarantee for wholly new descriptions. Reference: [14.09].

84. What would you improve first with more time and permitted complexity?

Classification quality is the bottleneck because the official score cannot exceed weighted F1. I would examine ambiguous labels, representation quality, and genuinely useful new information, using proper development evaluation. I would not first add arbitrary confidence transformations,

publish a private-score guarantee, or tune repeatedly on the final holdout.

85. What if the professor asks a result you did not calculate?

I should say that it was not measured, explain the appropriate calculation and its data requirements, and avoid inventing an answer. The project distinguishes verified outputs from plausible alternatives, approximate uncertainty checks, and unverified private/Mac outcomes.

86. Does a middle financial rank mean the observation has high probability density?

No. A value between two modes can have a middle rank despite low local density. Compatibility describes central rank, not density or a class likelihood. The confidence model evaluates this simple feature empirically; no kernel-density model or formal financial posterior is implemented. Reference: [10.18].

87. Can you score returned industry names directly against the integer-coded y_val?

No: the label representations must match. Either score `confidence_score(df_val.industry, names, confidence)` or first apply `le.transform(names)` and compare with `y_val`. A bijective renaming leaves the metric unchanged when applied consistently, but mixing strings and integer codes is not a valid comparison. References: [04], [14.04].

88. Does selecting the largest predicted probability directly optimize weighted F1?

No. With true posteriors, argmax maximizes expected per-record accuracy under equal error costs. Weighted F1 is a non-additive dataset metric. Logistic regression optimizes regularized log loss, and the small vote-weight comparison selects using the actual tuning F1. More elaborate class-specific decision rules would need their own development evidence and could overfit.

89. Is a model coefficient a causal effect?

No. It is an association conditional on the other chosen inputs, the training sample, and regularization. Correlated features can change coefficient size or sign. Even the availability coefficient cannot establish that collecting a missing statement would causally improve the same company's classification correctness.

90. Does this confidence capture every kind of uncertainty?

No. It estimates correctness from a small set of observable signals. It is not a complete decomposition of parameter uncertainty, irreducible ambiguity, unseen classes, or distribution shift. Additional techniques such as model refitting or prediction sets would have different assumptions and outputs; they are not claimed here.

91. Which pretrained text model produced the vectors?

The supplied embedding example uses `sentence-transformers/all-mpnet-base-v2`, whose output dimension matches these 768-coordinate vectors. The pickle alone does not certify its generation model or version, and several encoders share that dimension. I use the course-provided vectors rather than claiming to have regenerated or independently identified their provenance.

92. Why not divide a company's financial deviation by the standard error of an industry mean?

The event concerns one company's observation, not an average of n new observations. Dividing by $s/\sqrt{n}$ would make individual values appear more extreme simply because the reference group is large. Under appropriate normal assumptions an individual standardized distance uses the distribution's scale, while estimated prediction intervals require their own formula. This project instead uses empirical ranks without claiming a Gaussian individual-value test.

93. Would ROC AUC replace the confidence calibration analysis?

No. AUC measures ranking discrimination across thresholds, while Brier and the reliability diagram concern the numerical quality of probabilities. A ranking can remain unchanged after a monotonic confidence transformation that badly damages calibration. The skeleton's specified score and requested confidence-versus-accuracy analysis therefore remain the focus; no unimplemented AUC result is claimed.

A five-minute oral presentation

1. **Problem and inputs:** one of 25 industries from a supplied 768-coordinate business embedding; finances support correctness confidence. State the actual 31,706-record scope.
2. **Separation:** IDs stay together; fitting, tuning, calibration, and validation have distinct roles. Explain why learned transforms never fit evaluation data.
3. **Required methods:** PCA for a two-dimensional view and 128-dimensional k-NN input; LR keeps the full embeddings. State how they learn and how metrics differ.
4. **Measured behavior:** LR wins the required individual comparison; common and rare classes behave differently; shared errors limit ensemble diversity.
5. **Optional method:** tune a 25/75 probability vote, calculate predicted-industry financial rank compatibility and availability, then learn correctness confidence on held-out calibration records.
6. **Honest conclusion:** report 78.09% weighted F1, 67.50% official score, and the unmet 85% target. Explain review workload, repeated-representation limits, and the unverified private/Mac results.

Self-test without looking at the answers

- Draw the shape of each matrix used by the classifier and the confidence model.
- Explain why a raw probability vector has 25 columns but the confidence vector has only one entry per record.
- Calculate a confusion cell’s precision and recall using the worked class example.
- Derive the official-score ceiling and the Brier limit needed if F1 were 0.90.
- Trace one missing-finance observation through all five confidence inputs.
- Explain the negative compatibility coefficient without treating it as a causal result.
- Identify exactly which code lines would cause leakage if fitted on validation.
- Explain why the review group’s denominator cannot replace the official one.

Reproduction, resources, and data provenance

Running the analysis

Put the notebook in a project folder with `data/df_train.pkl` and `data/df_financials_train.pkl`. The PDF is study material and is not imported by Python. Start a fresh Python 3.12 Jupyter kernel and run every cell in order; a previous interactive session can hide missing definitions or stale objects. Relative data paths are resolved from the notebook working folder, not from this PDF.

The tested versions are NumPy 2.3.5, pandas 2.2.3, scikit-learn 1.8.0, SciPy 1.17.0, Matplotlib 3.10.8, seaborn 0.13.2, and JupyterLab 4.6.4. The course repository already has a matching requirements file. If these two documents are used by themselves, the same packages can be installed in a Python 3.12 virtual environment:

```
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install numpy==2.3.5 pandas==2.2.3 scikit-learn==1.8.0 scipy==1.17.0 matplotlib==3.10.8 seaborn==0.13.2
↪  jupyterlab==4.6.4
python -m jupyter lab
```

The first command creates an isolated environment; the second activates it; the third installs the libraries; the fourth opens Jupyter. These are setup commands, not additional analysis code to paste into a notebook cell.

Memory arithmetic and benchmark scope

Object	Approximate array-only storage
Fitting embeddings	78,892,032 bytes = 75.24 MiB
All four embedding sets together	97,400,832 bytes = 92.89 MiB
Fitting PCA coordinates	13,148,672 bytes = 12.54 MiB

One MiB is 2^{20} bytes and one GiB is 2^{30} bytes. The full runner-plus-kernel peak-memory upper bound was **1.37 GiB**. It adds process peak readings that need not occur at the same instant, so it is an upper bound rather than an exact concurrent total. It also does not measure every operating-system or unrelated application allocation. Original embedding strings, parsing temporaries, DataFrames, model objects, and figures explain why this exceeds a single array's size.

The measured fitting-related portion was 44.3 seconds and the complete fresh-kernel execution with verification 221.1 seconds on this Linux cloud instance. This is not an Intel Mac measurement. Current-instance timings can vary with resource load; repeating the run does not guarantee an identical duration. The CPU-only design and observed memory are compatible with a lightweight workflow, but hardware-specific compliance still requires the laptop run.

Verification performed

The real-data checks verified: every pair of company-ID sets is disjoint; feature dimensions and row-label alignment are correct; both classifier probability columns follow the same classes; the score reproduces the provided formula and rejects invalid confidence; output vectors have the requested shape and bounds; input reversal reverses predictions; deliberate target changes do not affect predictions; missing and infinite financial inputs remain usable; financial references have one row per fitting company. All 14 analysis code cells completed and produced seven plots. Tests and benchmark scripts remain outside the submitted notebook.

What belongs in GitHub

The requested deliverables are the notebook and this PDF. The existing requirements and ignore files can remain in the repository. Raw data, Python environments, experimental classifiers, and temporary verification files are excluded. Data publication rights are unconfirmed. A Git commit records a version; pushing sends it to the remote repository. A local file is not automatically a GitHub submission. Review the actual file list before any final publication.

Development checks and alternatives

These small checks used fitting and tuning companies. They explain practical choices; they are not evidence of a global optimum. Scores below are rounded because numerical reruns can differ in the last digits. Earlier feature experiments used some different base representations, so the table is not a factorial causal study of each individual change.

Check	Tuning weighted F1	Decision
LR, C=1	0.778	Useful standalone reference
LR, C=10	0.777	Stronger tested joint blend
LR, C=100	0.768	No improvement
LR, C=1000	0.737	No improvement
k-NN, PCA128, k=15	0.782	Retained representation
25/75 LR/PCA128-kNN vote	0.792	Retained simple ensemble
PCA128 cosine-distance k-NN	0.783	Did not improve the joint vote
Whitened PCA128 Euclidean k-NN	0.709	Excluded
Whitened PCA128 cosine k-NN	0.754	Excluded
RBF SVC, C=10, gamma=4	0.773	Excluded; fitting alone took 326 seconds
Text plus transformed finances/country LR	0.776	Extra label-model complexity not justified

Only measured alternatives are listed. Further RBF runs were stopped rather than presenting uncompleted trials as results. The selected notebook does not include an exhaustive search or a third industry classifier. The classification bottleneck remains unresolved at the 85% official-score target.

Complete per-industry results

Fitting counts describe model-training records; validation support describes the denominator of held-out class metrics. F1 values below are fractions. The same class order maps integer codes to industry names.

Industry	Fit	Val	LR F1	k-NN F1
Automobiles & Components	515	60	0.7049	0.7287
Banks	764	106	0.9717	0.9279
Capital Goods	2516	308	0.6474	0.6328
Commercial & Professional Services	666	82	0.3759	0.3363
Consumer Discretionary Distribution & Retail	551	57	0.5660	0.5586
Consumer Durables & Apparel	921	128	0.7549	0.7229
Consumer Services	546	58	0.7593	0.7895
Consumer Staples Distribution & Retail	151	18	0.4615	0.4286
Energy	1071	139	0.6950	0.6967
Equity Real Estate Investment Trusts (REITs)	348	51	0.8298	0.7312
Financial Services	1557	191	0.7646	0.7321
Food, Beverage & Tobacco	904	102	0.8426	0.8426
Health Care Equipment & Services	819	104	0.7633	0.7736
Household & Personal Products	189	19	0.4444	0.4651
Insurance	234	24	0.7826	0.8235
Materials	6797	824	0.9243	0.9248
Media & Entertainment	752	80	0.7595	0.7778
Pharmaceuticals, Biotechnology & Life Sciences	1563	204	0.8604	0.8610
Real Estate Management & Development	734	92	0.7363	0.7047

Industry	Fit	Val	LR F1	k-NN F1
Semiconductors & Semiconductor Equipment	497	63	0.6949	0.6838
Software & Services	1220	167	0.6941	0.7040
Technology Hardware & Equipment	1237	177	0.6627	0.6941
Telecommunication Services	181	22	0.6667	0.7500
Transportation	480	49	0.7879	0.7573
Utilities	468	46	0.7872	0.7143

Financial reference observation counts

These counts use unique fitting companies after within-ID financial aggregation and missing/infinite-value handling. They explain the actual denominator in a reference percentile; the total does not need to equal the number of fitting records because observations are missing.

Industry	Net margin	EBITDA margin	Asset turnover
Automobiles & Components	475	133	479
Banks	710	209	709
Capital Goods	2276	727	2305
Commercial & Professional Services	532	290	551
Consumer Discretionary Distribution & Retail	473	277	474
Consumer Durables & Apparel	832	272	833
Consumer Services	469	260	477
Consumer Staples Distribution & Retail	140	66	143
Energy	795	338	878
Equity Real Estate Investment Trusts (REITs)	323	221	325
Financial Services	1115	639	1293
Food, Beverage & Tobacco	782	310	790
Health Care Equipment & Services	678	411	708
Household & Personal Products	161	78	162
Insurance	211	121	212
Materials	5438	732	6193
Media & Entertainment	620	324	624
Pharmaceuticals, Biotechnology & Life Sciences	1074	572	1372
Real Estate Management & Development	636	253	643
Semiconductors & Semiconductor Equipment	457	102	464
Software & Services	1012	543	1034
Technology Hardware & Equipment	1143	308	1151
Telecommunication Services	154	87	162
Transportation	434	184	438
Utilities	414	193	424

Complete code listing

This is the same implementation as the notebook, with the short numbered comments. Plot outputs and prose observations remain in the notebook. Each reference has an explanation in its corresponding code-block chapter above.

Block 01: Imports and resources

```
import ast  # [01.01] Parse embedding strings safely as literal values.
from time import perf_counter  # [01.02] Measure elapsed time with a monotonic clock.
import numpy as np  # [01.03] Use numerical arrays and vector operations.
import pandas as pd  # [01.04] Load and join labeled tables.
import matplotlib.pyplot as plt  # [01.05] Create figures and label their axes.
import seaborn as sns  # [01.06] Draw the scatterplot, heatmaps, and boxplots.
from IPython.display import display  # [01.07] Display readable tables in Jupyter.
from sklearn.decomposition import PCA  # [01.08] Import the PCA implementation.
# [01.09] Use the same established estimator for industry and correctness models.
from sklearn.linear_model import LogisticRegression
# [01.10] Import the required classification and scoring tools.
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, f1_score
from sklearn.model_selection import GroupShuffleSplit  # [01.11] Split company groups without separating repeated IDs.
from sklearn.neighbors import KNeighborsClassifier  # [01.12] Import the neighbor classifier.
from sklearn.preprocessing import LabelEncoder  # [01.13] Map industry names to integer class codes.
from threadpoolctl import threadpool_limits  # [01.14] Control numerical-library CPU threads.
threadpool_limits(2)  # [01.15] Limit numerical-library threads to two.
started = perf_counter()  # [01.16] Start timing after imports and thread setup.
```

Block 02: Loading Data

```
# [02.01] Select the three supplied numerical financial ratios.
ratios = ['net_profit_margin', 'ebitda_margin', 'asset_turnover']
financials = pd.read_pickle('data/df_financials_train.pkl')  # [02.02] Read the supplied financial table.
# [02.03] Count missing and infinite values before cleaning.
display(pd.DataFrame({'Missing': financials[ratios].isna().sum(), 'Infinite': np.isinf(financials[ratios]).sum()}))
# [02.04] Check how many financial rows repeat a company ID.
print('Repeated financial IDs:', financials.id.duplicated().sum())
# [02.05] Treat infinity as missing; take one median per ratio and company.
financials = financials.replace([np.inf, -np.inf], np.nan).groupby('id')[ratios].median()
# [02.06] Keep every main record and attach at most one financial row per ID.
df = pd.read_pickle('data/df_train.pkl').merge(financials, on='id', how='left', validate='many_to_one')
```

Block 03: Separating the companies

```
def split(companies, size):  # [03.01] Reuse one short company-group splitting function.
    # [03.02] Choose one reproducible group split of the requested size.
    splitter = GroupShuffleSplit(n_splits=1, test_size=size, random_state=42)
    left, right = next(splitter.split(companies, groups=companies.id))  # [03.03] Take its two arrays of row positions.
    return companies.iloc[left], companies.iloc[right]  # [03.04] Select those rows without changing their order.

df_development, df_val = split(df, 0.1)  # [03.05] Reserve about ten percent of company IDs for validation.
df_fit, df_other = split(df_development, 0.1)  # [03.06] Separate model fitting from the remaining development records.
df_tune, df_cal = split(df_other, 0.5)  # [03.07] Divide the held-out development records into tuning and calibration.
parts = [df_fit, df_tune, df_cal, df_val]  # [03.08] Keep one consistent order for all feature and label sets.

def embeddings(companies):  # [03.09] Reuse the embedding conversion for existing and new companies.
    # [03.10] Parse the strings and store their coordinates as float32 arrays.
    return np.array(companies.business_description_embedding.apply(ast.literal_eval).tolist(), dtype=np.float32)

X_fit, X_tune, X_cal, X_val = [embeddings(part) for part in parts]  # [03.11] Convert each of the four sets once.
# [03.12] Show the actual record counts rather than assumed percentages.
print('Fitting, tuning, calibration, validation records:', [len(part) for part in parts])
print('Embedding coordinates:', X_fit.shape[1])  # [03.13] Show how many coordinates each embedding contains.
```

Block 04: Transforming Data (Label Encoding) and class distribution

```
le = LabelEncoder().fit(df_fit.industry)  # [04.01] Learn the industry-code mapping from fitting records only.
# [04.02] Check the skeleton requirement that held-out classes are known.
```

```
assert set(df_tune.industry).union(df_cal.industry, df_val.industry).issubset(le.classes_)
# [04.03] Apply the same mapping to every target vector.
y_fit, y_tune, y_cal, y_val = [le.transform(part.industry) for part in parts]
counts = df_fit.industry.value_counts()  # [04.04] Count industries in the actual classifier-fitting set.
# [04.05] Show counts and proportions without an additional plot.
display(pd.DataFrame({'Fitting records': counts, 'Share': counts / counts.sum()}))
```

Block 05: PCA — dimensionality reduction and visualization

```
fit_started = perf_counter()  # [05.01] Start timing PCA preparation.
pca = PCA(n_components=128, random_state=42)  # [05.02] Keep 128 components using the fixed random seed.
P_fit = pca.fit_transform(X_fit)  # [05.03] Learn PCA from fitting embeddings and project them.
# [05.04] Project the other sets without learning another PCA.
P_tune, P_cal, P_val = [pca.transform(X) for X in [X_tune, X_cal, X_val]]
fit_seconds = perf_counter() - fit_started  # [05.05] Store the PCA and transformation time.
fig, ax = plt.subplots(figsize=(10, 6))  # [05.06] Create a figure and axes for the required scatterplot.
# [05.07] Color fitting points by industry; small transparent points show overlap.
sns.scatterplot(x=P_fit[:, 0], y=P_fit[:, 1], hue=df_fit.industry.to_numpy(),
                hue_order=le.classes_, palette='husl', s=7, alpha=0.3, linewidth=0, ax=ax)
ax.set(xlabel='Principal component 1', ylabel='Principal component 2',  # [05.08] Label the two PCA coordinates and the
↪  plot.
       title='Fitting embeddings: first two PCA components')
# [05.09] Place the long industry legend outside the data region.
ax.legend(bbox_to_anchor=(1.02, 1), loc='upper left', fontsize=7, markerscale=3)
fig.tight_layout()  # [05.10] Fit the labels and legend within the figure.
plt.show()  # [05.11] Display the scatterplot.
# [05.12] Report the fractions of variance retained, not classification accuracy.
print('Variance retained by 2 and 128 components:',
      pca.explained_variance_ratio_[:2].sum(), pca.explained_variance_ratio_.sum())
```

Block 06: Fitting and comparing Classifier Models

```
lr = LogisticRegression(C=10, max_iter=1000)  # [06.01] Use the development-selected LR regularization and iteration
↪  limit.
# [06.02] Use fifteen distance-weighted neighbors with two parallel jobs.
knn = KNeighborsClassifier(n_neighbors=15, weights='distance', n_jobs=2)
fit_started = perf_counter()  # [06.03] Start timing the two classifier fits.
lr.fit(X_fit, y_fit)  # [06.04] Learn industry weights from the original fitting embeddings.
knn.fit(P_fit, y_fit)  # [06.05] Store fitting neighbors in the PCA representation.
fit_seconds += perf_counter() - fit_started  # [06.06] Add classifier fitting time to the PCA preparation time.
# [06.07] Report this measured part of the computation.
print('PCA, transforms, and classifier fitting seconds:', round(fit_seconds, 1))
# [06.08] Get both classifiers' probabilities on tuning companies.
lr_tune, knn_tune = lr.predict_proba(X_tune), knn.predict_proba(P_tune)
weights = [0.25, 0.5, 0.75]  # [06.09] Compare three simple logistic-regression voting weights.
# [06.10] Score each probability mixture using tuning weighted F1 only.
tuning_f1 = [f1_score(y_tune, (w * lr_tune + (1 - w) * knn_tune).argmax(1), average='weighted') for w in weights]
weight = weights[np.argmax(tuning_f1)]  # [06.11] Keep the weight with the highest tuning score.
display(pd.DataFrame({'LR weight': weights, 'Tuning weighted F1': tuning_f1}))  # [06.12] Show the small tuning
↪  comparison.
# [06.13] Cache calibration probabilities for the correctness models.
lr_cal, knn_cal = lr.predict_proba(X_cal), knn.predict_proba(P_cal)
# [06.14] Cache validation probabilities for all evaluations.
lr_val, knn_val = lr.predict_proba(X_val), knn.predict_proba(P_val)
# [06.15] Take the highest-probability industry code for each base model.
predictions = {'Logistic regression': lr_val.argmax(1), 'k-NN': knn_val.argmax(1)}
```

Block 07: Classification results and errors

```
rows = []  # [07.01] Collect the two models' summary metrics.
for name, prediction in predictions.items():  # [07.02] Analyze both classifiers with the same short loop.
    # [07.03] Calculate accuracy and weighted/macro F1 on the same records.
    rows.append([name, accuracy_score(y_val, prediction),
                 f1_score(y_val, prediction, average='weighted'),
                 f1_score(y_val, prediction, average='macro')])
    # [07.04] Get per-industry precision, recall, F1, and support.
    report = pd.DataFrame(classification_report(y_val, prediction,
        labels=np.arange(len(le.classes_)), target_names=le.classes_,
        output_dict=True, zero_division=0)).T.loc[le.classes_]
    # [07.05] Align industry names when relating fitting size to validation errors.
```

```
    print(name, '| fitting size / error-rate correlation:', round(counts.corr(1 - report['recall']), 3))
    display(report.sort_values('f1-score'))  # [07.06] Show the weakest industries first.
    # [07.07] Count true-industry rows against predicted-industry columns.
    cm = pd.DataFrame(confusion_matrix(y_val, prediction, labels=np.arange(len(le.classes_))),
                      index=le.classes_, columns=le.classes_)
    fig, ax = plt.subplots(figsize=(11, 9))  # [07.08] Create axes for this model's confusion heatmap.
    # [07.09] Normalize each true-industry row before coloring it.
    sns.heatmap(cm.div(cm.sum(axis=1), axis=0), cmap='Blues', vmin=0, vmax=1, ax=ax)
    # [07.10] Label the true and predicted axes explicitly.
    ax.set(xlabel='Predicted industry', ylabel='True industry', title=name + ': fraction of each true industry')
    fig.tight_layout()  # [07.11] Keep the long industry labels visible.
    plt.show()  # [07.12] Display the confusion heatmap.
    np.fill_diagonal(cm.values, 0)  # [07.13] Exclude correct counts from the error-pair ranking.
    # [07.14] Show the five largest off-diagonal confusion counts.
    display(cm.stack().nlargest(5).rename('Misclassifications'))
# [07.15] Build the direct classifier comparison table.
comparison = pd.DataFrame(rows, columns=['Model', 'Accuracy', 'Weighted F1', 'Macro F1']).set_index('Model')
display(comparison)  # [07.16] Display the comparison.
# [07.17] Count records where the models are correct or wrong together.
display(pd.crosstab(predictions['Logistic regression'] == y_val, predictions['k-NN'] == y_val,
                    rownames=['LR correct'], colnames=['k-NN correct']))
```

Block 08: Optional: Confidence Weighted Prediction — official score

```
def confidence_score(y_true, y_pred, confidence):  # [08.01] Keep the course's confidence-score interface and formula.
    """Official weighted F1 multiplied by one minus correctness Brier."""
    y_true = np.asarray(y_true)  # [08.02] Allow true labels to be supplied as a list or array.
    y_pred = np.asarray(y_pred)  # [08.03] Allow predicted labels to be supplied as a list or array.
    confidence = np.asarray(confidence, dtype=float)  # [08.04] Use numerical floating-point confidence values.
    # [08.05] Require one prediction and confidence for every target.
    assert len(y_true) == len(y_pred) == len(confidence), 'Inputs must have the same length'
    # [08.06] Require every confidence to lie between zero and one.
    assert np.all((confidence >= 0) & (confidence <= 1)), 'Confidence must be between 0 and 1'
    # [08.07] Calculate class-support-weighted F1; confidence is not a weight.
    f1 = f1_score(y_true, y_pred, average='weighted')
    correct = (y_true == y_pred).astype(float)  # [08.08] Represent a correct prediction as one and an error as zero.
    brier = np.mean((confidence - correct) ** 2)  # [08.09] Average the squared errors of the correctness confidence.
    return f1 * (1 - brier)  # [08.10] Apply exactly the official multiplicative score.
```

Block 09: Financial distributions

```
profile_data = df_fit.drop_duplicates('id')  # [09.01] Give each fitting company one vote in the financial references.
# [09.02] Show observation counts and medians for every industry and ratio.
display(profile_data.groupby('industry')[ratios].agg(['count', 'median']))
for column in ratios:  # [09.03] Inspect all three ratios with the same plotting code.
    fig, ax = plt.subplots(figsize=(9, 7))  # [09.04] Create readable axes for this ratio.
    # [09.05] Show quartiles and medians; hide dots rather than delete outliers.
    sns.boxplot(data=profile_data, x=column, y='industry', showfliers=False, ax=ax)
    ax.set_xscale('symlog', linthresh=0.1)  # [09.06] Keep negative values while displaying large magnitudes.
    # [09.07] Label the financial ratio, industry, and plot title.
    ax.set(xlabel=column.replace('_', ' '), ylabel='Industry',
           title='Fitting financial distributions: ' + column.replace('_', ' '))
    fig.tight_layout()  # [09.08] Prevent labels from being clipped.
    plt.show()  # [09.09] Display this financial distribution.
```

Block 10: Text votes and financial compatibility

```
profiles = {}  # [10.01] Store the fitting-only financial reference samples.
for industry, group in profile_data.groupby('industry'):  # [10.02] Separate the unique fitting companies by known
↪  industry.
    for column in ratios:  # [10.03] Create one reference for each of the three ratios.
        # [10.04] Sort available values so percentile lookups are quick.
        profiles[industry, column] = np.sort(group[column].dropna().to_numpy())

def confidence_features(companies, proba_lr, proba_knn):  # [10.05] Reuse exactly one confidence-feature calculation.
    # [10.06] Combine aligned industry probabilities with the tuning-selected weight.
    votes = weight * proba_lr + (1 - weight) * proba_knn
    prediction = votes.argmax(1)  # [10.07] Choose each row's largest ensemble vote.
    top = np.sort(votes, axis=1)  # [10.08] Sort each vote vector to find its largest two entries.
    agreement = proba_lr.argmax(1) == proba_knn.argmax(1)  # [10.09] Record whether LR and k-NN choose the same
↪  industry.
```

```
    names = le.inverse_transform(prediction)  # [10.10] Find the industry names predicted by the ensemble.
    # [10.11] Add absent financial columns as missing rather than inventing values.
    financial = companies.reindex(columns=ratios)
    # [10.12] Start each company's compatibility total and evidence count at zero.
    total, available = np.zeros(len(companies)), np.zeros(len(companies))
    for (industry, column), reference in profiles.items():  # [10.13] Visit each industry-ratio reference once.
        values = financial[column].to_numpy()  # [10.14] Read this ratio in the input-row order.
        # [10.15] Use finite values for the predicted industry, never the true industry.
        mask = (names == industry) & np.isfinite(values)
        # [10.16] Count reference values strictly below the observation.
        left = np.searchsorted(reference, values[mask], side='left')
        # [10.17] Count reference values below or equal to the observation.
        right = np.searchsorted(reference, values[mask], side='right')
        # [10.18] Use the midpoint of the two empirical cumulative fractions.
        percentile = (left + right) / (2 * len(reference))
        # [10.19] Add a bounded middle-versus-tail compatibility score.
        total[mask] += 2 * np.minimum(percentile, 1 - percentile)
        available[mask] += 1  # [10.20] Count this observed financial contribution.
    # [10.21] Average available scores; distinguish no evidence with availability zero.
    compatibility = np.divide(total, available, out=np.full(len(companies), 0.5), where=available > 0)
    # [10.22] Return the class codes and five confidence inputs in a fixed order.
    return prediction, np.column_stack([top[:, -1], top[:, -1] - top[:, -2],
                                        agreement, compatibility, available / len(ratios)])
```

Block 11: Learning confidence

```
# [11.01] Build confidence inputs from held-out calibration companies.
pred_cal, features_cal = confidence_features(df_cal, lr_cal, knn_cal)
correct_cal = (pred_cal == y_cal).astype(int)  # [11.02] Set the binary learning target to whether the ensemble was
↪  correct.
# [11.03] Fit the text-only confidence comparison using the first three inputs.
text_calibrator = LogisticRegression().fit(features_cal[:, :3], correct_cal)
# [11.04] Fit correctness confidence with the two additional financial inputs.
finance_calibrator = LogisticRegression().fit(features_cal, correct_cal)
# [11.05] Build the same inputs for validation without fitting anything.
pred_val, features_val = confidence_features(df_val, lr_val, knn_val)
```

Block 12: Evaluating confidence

```
confidences = {  # [12.01] Compare raw votes, text confidence, and financial confidence.
    'Raw ensemble': features_val[:, 0],
    'Text calibration': text_calibrator.predict_proba(features_val[:, :3])[:, 1],
    'Financial calibration': finance_calibrator.predict_proba(features_val)[:, 1]
}
rows = []  # [12.02] Collect all three approaches using the same predictions.
for name, confidence in confidences.items():  # [12.03] Evaluate each confidence vector without changing its labels.
    # [12.04] Calculate weighted F1, correctness Brier, and the official score.
    rows.append([name, f1_score(y_val, pred_val, average='weighted'),
                 np.mean((confidence - (pred_val == y_val)) ** 2),
                 confidence_score(y_val, pred_val, confidence)])
# [12.05] Give the confidence comparison clear column names.
bonus = pd.DataFrame(rows, columns=['Method', 'Weighted F1', 'Brier', 'Official score']).set_index('Method')
display(bonus)  # [12.06] Display the complete confidence comparison, including weaker results.
```

Block 13: Confidence and human review

```
# [13.01] Use the financial-calibrated confidence for inference and review.
confidence_val = confidences['Financial calibration']
# [13.02] Pair confidence with observed correctness for evaluation only.
outcomes = pd.DataFrame({'Confidence': confidence_val, 'Correct': pred_val == y_val})
# [13.03] Divide zero to one into ten confidence intervals.
outcomes['Bin'] = pd.cut(confidence_val, np.linspace(0, 1, 11), include_lowest=True)
# [13.04] Compare mean confidence, actual accuracy, and sample size in each bin.
reliability = outcomes.groupby('Bin', observed=True).agg(
    Count=('Correct', 'size'), Mean_confidence=('Confidence', 'mean'), Accuracy=('Correct', 'mean'))
display(reliability)  # [13.05] Show counts so small confidence bins are not overinterpreted.
fig, ax = plt.subplots()  # [13.06] Create the confidence-versus-accuracy plot.
# [13.07] Draw the ideal calibration line, where confidence equals accuracy.
ax.plot([0, 1], [0, 1], '--', color='gray', label='Perfect calibration')
# [13.08] Plot the observed validation-bin values.
```

```
ax.plot(reliability.Mean_confidence, reliability.Accuracy, 'o-', label='Validation bins')
# [13.09] Label the calibration axes and title.
ax.set(xlabel='Mean predicted confidence', ylabel='Observed accuracy', title='Confidence versus validation accuracy')
ax.legend()  # [13.10] Identify the observed and ideal lines.
fig.tight_layout()  # [13.11] Fit the plot labels within the figure.
plt.show()  # [13.12] Display the calibration plot.
review = confidence_val < 0.8  # [13.13] Flag confidence strictly below the fixed 0.8 review threshold.
# [13.14] Compare flagged and unflagged records; True means human review.
display(outcomes.groupby(review).agg(Count=('Correct', 'size'), Accuracy=('Correct', 'mean')))
# [13.15] Show review workload and the number of errors the flags capture.
print('Review fraction:', review.mean(), '| errors captured:',
      np.sum(review & (pred_val != y_val)), 'of', np.sum(pred_val != y_val))
```

Block 14: Predicting new companies and checking repeated representations

```
def predict_industry(companies):  # [14.01] Expose the required label-vector and confidence-vector prediction
↪  interface.
    X = embeddings(companies)  # [14.02] Convert new descriptions with the same embedding parser.
    # [14.03] Reuse both classifiers, fitting PCA, and the confidence features.
    prediction, features = confidence_features(companies, lr.predict_proba(X), knn.predict_proba(pca.transform(X)))
    # [14.04] Return industry names and correctness confidence in input-row order.
    return le.inverse_transform(prediction), finance_calibrator.predict_proba(features)[:, 1]

example = df_val.drop(columns='industry').head(3)  # [14.05] Demonstrate prediction after removing the true industry
↪  column.
labels, confidence = predict_industry(example)  # [14.06] Call the same public prediction function used for new
↪  companies.
# [14.07] Display three real examples with their IDs and predicted outputs.
display(pd.DataFrame({'ID': example.id.to_numpy(), 'Predicted industry': labels, 'Confidence': confidence}))
# [14.08] Check exact stored embedding strings shared with fitting records.
seen = df_val.business_description_embedding.isin(df_fit.business_description_embedding)
# [14.09] Compare accuracy on seen and unseen stored representations.
display(pd.DataFrame({'Embedding seen in fitting': seen.to_numpy(), 'Correct': pred_val == y_val})
        .groupby('Embedding seen in fitting').agg(Count=('Correct', 'size'), Accuracy=('Correct', 'mean')))
# [14.10] Report elapsed notebook time through the final code cell.
print('Whole notebook seconds:', round(perf_counter() - started, 1))
```

Sources and further reading

Assignment and project sources

- Current-year `code_skeleton_industry_classification.ipynb`: task structure, required model comparison, tuple output, confidence bounds, and official scoring formula.
- Supplied `df_train.pkl` and `df_financials_train.pkl`: the actual company records and financial ratios.
- The student's Foundations of Data Science project brief: minimum-code priority, financial/uncertainty goals, 85% target, laptop constraint, and requirement to understand every line.
- Prior-year materials: background only; their different confidence scoring is not used.

Official library documentation

- Python literal parsing
- NumPy arrays and indexing
- pandas joins
- GroupShuffleSplit
- LabelEncoder
- PCA
- LogisticRegression
- KNeighborsClassifier
- Classification metrics
- Probability calibration concepts

These documentation links support API and conceptual reading. The exact score is defined by the course function, rather than inferred from a generic documentation example. Figures and numerical project results in this guide come from the executed notebook; illustrative arithmetic examples are explicitly identified.